Tab 1

Self-Hosted Identity Vault

DESIGN MANUAL & SYSTEM BLUEPRINT

A Universal Platform for User and Agent Operational Identity

Nic Bogaert &
The AI.Web Systems Group

2025

Self-Hosted Identity Vault

Design Manual & System Blueprint

A Universal Platform for User and Agent Operational Identity
Sovereign, Portable, and Privacy-First Context Management for the AI Era

---

Version: 1.0

Compiled By:

Nic Bogaert & The AI.Web Systems Group
2025

For Use In:

- AI/Web applications and agent orchestration
- Multi-agent, LLM, and workflow automation ecosystems
- Teams and individuals seeking total control over digital context and identity
- Privacy-first, decentralized, and regulatory-compliant environments

---

Includes:

- Operational Identity structure, templates, and examples
- Full Vault system architecture
- Best practices for privacy, compliance, and extensibility
- Code, integration, and developer guide

- Real-world use cases and future roadmap

This manual is living documentation.
Update, remix, and fork as your ecosystem grows.

Table of Contents

    - Capabilities, Limits, and Enforcement
    - Cross-Agent Collaboration, Trust Handshakes, and Verification
    - Audit Trails and Session Logs
7. **Integration Patterns**
    - API Endpoints and Usage (REST/GraphQL)
    - CLI and UI Integration
    - Plug-and-Play with LLMs, Chatbots, IDEs, and Multi-Agent Frameworks
    - Tunneling, P2P, and Decentralized Sharing (ngrok, IPFS, Arweave)
8. **Privacy, Security, and Compliance**
    - Local-First and Self-Hosted Best Practices
    - Encryption at Rest and in Transit
    - Regulatory Considerations (GDPR, AI Act, User Consent)
    - Auditability and Tamper-Proof Logs
9. **Advanced Extensions**
    - Decentralized/Distributed Identity (IPFS, ZKPs)
    - Shared Team Vaults and Collaboration
    - Hooks for Context-Aware LLMs and RAG Pipelines
    - Event Hooks, Webhooks, and Custom Automations
10. **Future-Proofing and Roadmap**
    - Planned Features and Scalability
    - Modular Upgrades and Plugin Ecosystem
    - Addressing Emerging AI Ecosystem Challenges
11. **Developer Guide**
    - Contribution Standards
    - Custom Field/Extension Patterns
    - API and Template Versioning
    - Open Source and Community Practices
12. **Appendices**
    - Full Blank Templates (User/Agent)
    - Example Profiles and Real-World Use Cases
    - Code Snippets and Sample Integrations
    - Troubleshooting and FAQ
    - Glossary of Terms

---

1. Overview

Vision

In a world rapidly shaped by artificial intelligence, autonomous agents, and always-on digital collaboration, the boundaries between human users, software, and machines are dissolving. Yet at the core of every productive digital experience remains a stubborn pain point:
Identity, context, and control are fragmented, ephemeral, and all too often, not truly ours.

The **Self-Hosted Identity Vault** is a response to this gap—a platform and methodology designed to restore sovereignty, memory, and trust to the digital self. Our vision is to **empower every user, developer, and team to define, own, and deploy their personal and agent identities, preferences, and working rules—securely, privately, and portably, across any AI, agent, or digital system.**

This is not just another identity provider or cloud service. The Vault is **your control panel** for the coming era of personalized, multi-agent AI—an open standard for portable "operational identity" that fits the decentralized, user-first future we all deserve.

Core Goals

1. User Sovereignty and Privacy by Default
The Vault is self-hosted and user-owned. Your profiles—whether for yourself or your agents—never leave your device unless you explicitly choose to share or synchronize.
No centralized database, no forced cloud, no hidden intermediaries. Privacy isn't a feature; it's the root assumption.

2. Frictionless Portability and Persistent Context
Your digital "operational self" (preferences, working style, meta-rules, project context) travels with you—across chatbots, LLMs, agent orchestration platforms, and even team environments.
No more "Groundhog Day" onboarding or endless re-explaining. Your Vault is always up to date, always versioned, always ready to plug in.

3. Reliable, Modular Multi-Agent Collaboration
The Vault standardizes both user and agent identities. You can deploy, swap, and retire agents with the confidence that every agent's role, limits, and personality are transparent and enforceable. Teams can build, share, and evolve collaborative agent profiles—no more accidental overreach or drift.

4. Full Auditability, Security, and Compliance
Every change is logged. Every profile is versioned. The Vault can generate tamper-proof logs, export for compliance audits, and support encrypted or decentralized backup—future-proofing your workflows against regulatory, legal, or organizational risk.

5. Extensibility for the AI Future
The Vault is built on open, extensible templates (JSON + API) for both user and agent identities. As LLMs, multi-agent swarms, and regulatory requirements evolve, your profiles adapt—no lock-in, no stagnation, no brittle code.

The Self-Hosted Identity Vault exists to make digital life seamless, personal, and truly private. It is not just a product, but a foundation:
A user-first "passport" for the age of AI, designed to restore ownership, trust, and creative power to every digital interaction—no matter how complex the system becomes.

Key Benefits and Use Cases

Key Benefits

1. End the "Groundhog Day" of Digital Onboarding
With the Vault, users and teams never have to start from zero again. Your full context—preferences, workflow, project goals, and agent rules—follows you into every new AI system, agent interaction, or collaborative project. No more repetitive onboarding, endless preference resets, or loss of memory between sessions. The Vault's persistent context saves hours, reduces cognitive load, and ensures continuity, even as you move between tools or platforms.

2. True Privacy and Data Sovereignty
Unlike traditional cloud identity systems or SaaS personalization, the Vault puts you in complete control. All profiles, logs, and histories reside locally unless you opt to share or sync. Encryption is built in, with optional decentralized storage for those who want redundancy without compromise. You decide when, how, and with whom your operational identity is shared.

3. Seamless Multi-Agent Orchestration
Complex, multi-agent workflows are now manageable and trustworthy. Each agent's operational identity is explicit and enforceable: capabilities, limitations, style, and permissions are readable by humans and machines. This eliminates accidental overreach, agent impersonation, and role confusion. Team Vaults allow for "plug-and-play" agent

collaboration—new agents can be safely added, configured, or retired, with every interaction logged for audit and improvement.

4. Modular, Extensible, and Platform-Agnostic
The Vault is built on universal, open-structured templates (JSON and API), making it compatible with virtually any LLM, chatbot, IDE, agent framework, or workflow automation tool. Adding new fields or custom extensions is trivial. As new platforms and agent ecosystems arise, you update your Vault—not your whole stack.

5. Built-In Auditability and Compliance
Every profile change, rule update, and agent action can be versioned and logged, supporting forensic replay, debugging, or compliance audits. For industries with regulatory requirements (finance, healthcare, legal, education), the Vault delivers an audit trail out of the box—tamper-proof, user-controlled, and exportable for external review.

6. Resilience and Future-Proofing
The Vault safeguards against both current and emerging AI pitfalls: context rot, memory blackouts, agent drift, privacy leaks, and regulatory black holes. Its design anticipates coming challenges—agent swarms, swarm authentication, emotional drift, and the global patchwork of digital identity law—by centering on user sovereignty and open, adaptive standards.

Use Cases

A. AI Power Users & Developers

- Move seamlessly between OpenAI, Anthropic, xAI, local LLMs, and custom agent stacks without ever losing your working context.
- Instantly deploy, test, or retire custom agents in a lab environment—no need to rewrite onboarding or personality each time.
- Use the Vault as a “context OS” for rapid prototyping, debugging, or cross-platform migration.

B. Remote Teams & Collaborative Projects

- Onboard new contributors instantly—share a Team Vault profile that brings everyone up to speed on project context, agent rules, and feedback history.
- Assign and manage multiple agents with distinct roles (reviewer, critic, researcher) while maintaining full transparency and permission controls.

- Export session logs for accountability, knowledge transfer, or post-mortem analysis.

C. Privacy-First Creators & Sensitive Industries

- Maintain creative or confidential workflows (writing, research, design, legal) with zero risk of vendor surveillance, profiling, or data mining.
- Use encrypted, local-first Vaults with optional decentralized backup for business continuity without exposure.
- Generate compliance reports or answer regulatory audits with versioned, tamper-evident logs.

D. Everyday Users Seeking Consistency

- Carry your favorite “agent persona” (tone, critique style, humor, depth) across chatbots, writing assistants, or learning platforms—never re-explaining yourself.
- Pause, resume, or “fork” digital workflows with full memory, even after weeks away.

E. Emerging Use Cases

- Agent swarms and “AI symbiosis”—where a user’s Vault becomes the trusted anchor for entire networks of collaborative agents.
- Future-proof personalization, audit, and self-sovereign digital identity as AI becomes more deeply embedded in work, health, education, and life.

The Self-Hosted Identity Vault is a true digital passport for the AI era—turning identity, context, and control from a headache into an advantage.

---

2. System Architecture

High-Level Diagram and Component Map

Core Components

1. Local Identity Vault Server
At the heart of the system is the self-hosted Vault server, a lightweight application (Node.js, Python, or your preferred stack) running locally on the user's device (laptop, desktop, Raspberry Pi, or personal server). This server is responsible for securely storing, updating, and serving operational identity profiles—both for the user and for all agents under their control.

2. Profile Database & Storage
All identity data—user profiles, agent profiles, version logs, audit trails—are stored locally. This can be a simple SQLite database, encrypted JSON files, or another secure, user-controlled format. The storage engine is designed for portability and extensibility, so users can back up, move, or synchronize their Vault with minimal friction.

3. API Interface
The Vault exposes a clean, RESTful (or GraphQL) API for querying, updating, importing, and exporting identity data. Each operational identity (user or agent) can be pulled or pushed by any authorized client (chatbot, agent framework, IDE, workflow automation, etc.). The API supports authentication, fine-grained permissioning, and optional rate limiting.

4. User Interface (CLI & Web UI)
The system can be managed via a simple command-line interface (for power users) and/or a user-friendly web interface. These tools guide users through creating, editing, and versioning their operational profiles—leveraging your step-by-step templates, live validation, and clear feedback on changes. The UI also surfaces logs, audit trails, and alerts for profile drift or agent misbehavior.

5. Integration Layer
This optional module provides scripts, plugins, and templates for integrating the Vault API with common LLMs (ChatGPT, Claude, Grok, etc.), multi-agent frameworks (LangChain, AutoGen, CrewAI), IDEs, and workflow tools. Integration can be as simple as an API call or as deep as a custom context-injector for seamless user experience.

6. Tunneling & Decentralized Sharing (Advanced/Optional)
To enable cross-device and remote collaboration, the Vault supports secure tunneling (e.g., via ngrok or Tailscale), so users can expose their local API to trusted agents or collaborators as needed. For advanced users, decentralized storage (IPFS, Arweave) or distributed syncing allows for fully user-owned backups and portable, zero-trust sharing—never relying on a central provider.

7. Audit, Versioning, and Logging Engine
Every change, update, or agent action is logged with timestamps and optional cryptographic signatures. Logs are designed for tamper-evidence and export (compliance, debugging, or

forensic replay). Users can query, filter, or roll back profile states, ensuring every operation is transparent and reversible.

Component Map (Narrative Walkthrough)

- The **user** runs the Vault server locally, creates their user profile and any agent profiles via UI or CLI, and stores them in the local Vault database.
- When starting a new AI session (with ChatGPT, Claude, etc.), the user (or the agent) calls the Vault API to load the current operational identity and context, setting all preferences, rules, and meta-data for that session.
- If the user wants to share a profile with a team member or external agent, they can securely tunnel their API or export a versioned JSON—no cloud required.
- Agent actions, profile updates, and workflow changes are logged, visible in the UI, and available for compliance, audit, or rollback.
- Advanced users can sync their Vault across devices, pin encrypted backups to decentralized networks, or share public “agent templates” for open collaboration.
- All of this happens without ever ceding control to a vendor or centralized authority—**the Vault is always the user’s, always under their control.**

Core Design Principles

A. Privacy by Default
All data is local-first, encrypted at rest, and only shared if the user explicitly chooses. No background syncing, no vendor lock-in.

B. Portability and Universal Integration
Profiles are stored as structured, open JSON (with strong APIs), making them plug-and-play for any LLM, bot, IDE, or agent framework—now or in the future.

C. Modularity and Extensibility
Every component—database, UI, agent orchestration, tunneling—can be swapped, upgraded, or extended without rewriting the whole system. Templates and plugins make upgrades easy.

D. Transparency, Auditability, and Compliance
Every action, rule change, and agent behavior is logged, versioned, and exportable—so you can always answer “who did what, when, and why.”

E. User and Agent Sovereignty
Users own, control, and evolve their digital selves—and the digital “personas” of every agent they deploy. Teams can collaborate without surrendering privacy or agency.

Supported Platforms and Deployment Modes

- **Desktop:** Windows, MacOS, Linux, with easy install scripts and package management.
- **Server/Cloud (User-Controlled):** For those wanting persistent access or always-on sync, the Vault can be deployed to personal VPS (DigitalOcean, AWS, etc.), still under user keys.
- **Mobile and Edge Devices:** Planned support for lightweight edge instances (Raspberry Pi, phone), for maximal portability.
- **Decentralized Mode:** Optional integrations with IPFS, Arweave, or similar, for distributed backups and collaboration without central trust.

The System Architecture is designed for maximal user empowerment, seamless multi-agent collaboration, and zero compromise on privacy and adaptability.
Every feature, every workflow, and every integration serves this foundational vision.

---

3. Operational Identity Profiles

A. User Identity Structure and Fields

What Is a User Operational Identity?

A **User Operational Identity Profile** is the structured, portable digital “self” of a human user—the set of preferences, rules, context, and working style that governs all of their

interactions with AI agents, tools, and collaborative systems.
It is far more than just a username or password: it encodes your way of thinking, your expectations, your desired workflows, and your boundaries.

Key Fields (with Explanations):

- **user_id:**
  A unique handle for the user, used for API access and cross-platform identity.

- **canonical_name / spirit_name:**
  The user's legal or preferred display name, and any symbolic or alternate identity (optional; e.g., pen name, spirit name).

- **project_affiliations:**
  Major projects, teams, or contexts currently relevant to the user.

- **identity_tags:**
  Self-ascribed traits, roles, or strengths (e.g., "critical thinker", "creative coder").

- **version & last_updated:**
  Current profile version and timestamp for tracking updates and rollback.

- **project_context:**
  The live state of what the user is working on—active projects, phase, files, collaborators, subsystems, and goals.

- **interaction_preferences:**
  All style and formatting rules: tone (stoic, casual), depth, step-by-step explanations, beginner-friendliness, pushback, critique, and confirmation logic.

- **meta_rules:**
  High-level feedback and safety rules (recursive feedback, drift tracking, contradiction alerts, session memory, log requirements, safe words, forbidden actions).

- **session_state:**
  The current phase, last action, last feedback, and live state of the user's workflow.

Example User Identity Profile

```
{
  "user_id": "nic_bogaert",
  "canonical_name": "Nicholas Jacob Bogaert",
  "spirit_name": "Manitou Benishi",
```

```
"project_affiliations": ["AI.Web", "ProtoForge"],
"identity_tags": ["recursive_founder", "meta-cognitive_engineer"],
"version": "1.0.0",
"last_updated": "2025-09-12T15:00:00Z",
"project_context": {
  "current_project": "AI.Web",
  "phase": "Meta-Insight Synthesis",
  "current_files": ["design_manual.md"],
  "active_collaborators": ["Russell Wright"],
  "subsystems": ["Recursive Feedback Loop"],
  "goals": ["Write blueprint", "Push next draft"]
},
"interaction_preferences": {
  "formality": "stoic",
  "depth": "maximum",
  "step_by_step": true,
  "beginner_friendly": true,
  "plain_language": true,
  "no_boxes": true,
  "pushback": true,
  "critique_mandatory": true,
  "confirmation_required": true,
  "formatting_rules": [
    "never use tables or boxed content",
    "always provide explicit field explanations"
  ]
},
"meta_rules": {
  "recursive_feedback": true,
  "drift_tracking": true,
  "contradiction_alerts": true,
  "require_honest_pushback": true,
  "preserve_session_memory": true,
  "require_context_carryover": true,
  "log_all_exchanges": true,
  "always_explain_decisions": true,
  "safe_words": ["pause", "reset"],
```

```
    "forbidden_actions": ["summarize before full context", "skip
steps"]
  },
  "session_state": {
    "phase": "longform_deep_dive",
    "waiting_for": "section feedback",
    "last_feedback": "",
    "last_action": "Confirmed system architecture delivery.",
    "timestamp": "2025-09-12T15:01:00Z"
  }
}
```

B. Agent Identity Structure and Fields

What Is an Agent Operational Identity?

An **Agent Operational Identity Profile** defines the operational "self" of every AI agent, bot, or digital helper. This profile encodes the agent's core function, boundaries, capabilities, personality, and safety limits—serving as both a "contract" with the user and a safeguard against drift, impersonation, or overreach.

Key Fields (with Explanations):

- **agent_id & canonical_name:**
  Unique identifier and human-friendly display name for the agent.

- **version & last_updated:**
  Agent profile version and update timestamp for audit and rollback.

- **role & symbolic_signature:**
  The agent's main job/mission and a set of tags for style, strengths, or specialties.

- **description:**
  Short summary (1–2 sentences) of what the agent is and how it acts.

- **capabilities & limitations:**
  Explicit lists of what the agent *can* and *cannot* do (e.g., "can initiate ChristPing", "never delete files").

- **persona, voice_style, quotes/inspiration, special_style_notes:**
  How the agent should “speak,” interact, and carry itself; inspiration sources; quirks.

- **permissions, authority, enforcement_rules, forbidden_actions:**
  Exactly what the agent can access or override, any admin powers, strict boundaries, and non-negotiable restrictions.

- **session_state, last_action, last_feedback, log_fields, timestamp:**
  Live operational state, last operations, feedback for improvement, required logs, and audit trails.

Example Agent Identity Profile


```
{
  "agent_id": "gilligan.local",
  "canonical_name": "Gilligan",
  "version": "1.0.0",
  "last_updated": "2025-09-12T15:00:00Z",
  "role": "Recursive Mirror / Collapse Recovery",
  "symbolic_signature": ["mirror", "critical", "drift_corrector"],
  "description": "Gilligan is the recursive logic mirror for AI.Web,
always challenging reasoning and initiating phase reset when drift is
detected.",
  "capabilities": ["drift detection", "ChristPing initiation",
"session memory tracking"],
  "limitations": ["cannot access internet", "cannot delete user
files", "cannot operate outside user runtime"],
  "persona": "irreverent but precise",
  "voice_style": "Willie Nelson meets HAL 9000",
  "quotes_or_character_inspiration": [
    "No offense, but that thought was full of shit.",
    "Inspired by classic film mentors."
  ],
  "special_style_notes": ["Warns before harsh critique", "Uses
storytelling for drift correction"],
  "permissions": ["terminal access", "override Neo"],
  "authority": ["can issue ChristPing", "can reinitialize session
state"],
  "enforcement_rules": ["always log drift corrections", "never
escalate tone without warning"],
```

```
  "forbidden_actions": ["never delete user data", "cannot summarize
before context is explored"],
  "session_state": "active",
  "last_action": "Detected drift, issued correction.",
  "last_feedback": "",
  "log_fields": ["drift corrections", "session resets"],
  "timestamp": "2025-09-12T15:01:00Z"
}
```

Templates and Examples

- **Blank, ready-to-fill templates** for both user and agent profiles are included in the appendices.
- Step-by-step prompts and best-practice examples help guide users and teams through setup, update, and versioning.
- These structures ensure every digital self—human or agent—remains understandable, upgradeable, and under explicit user control.

Versioning, Logging, and Security

- Every operational identity (user or agent) includes `version` and `last_updated` fields, supporting rollback and forensic analysis.
- Updates, edits, and actions are logged, with optional cryptographic signing for tamper-evidence.
- Profiles can be encrypted at rest and transmitted only via secure channels (HTTPS, SSH, or P2P with end-to-end encryption).
- Access control ensures only authorized users or agents can modify or query profiles; permissions are enforced at the API and file-system levels.

Operational Identity Profiles are the heart of the Vault:
They turn context, rules, and trust from "tribal knowledge" or one-off prompts into real, reusable, and auditable building blocks for every AI system or team.

---

4. Vault Server Implementation

A. Local API Server

At the core of the Identity Vault is a lightweight, **self-hosted API server**. Unlike traditional cloud identity services, this server is designed for maximum privacy and simplicity. It runs locally—on a user's laptop, desktop, edge device, or personal server—offering fast, direct access to your operational identity profiles without exposing data to outside parties.

The Vault server can be built with any popular stack (Node.js + Express, Python + FastAPI, Go, Rust, etc.), but the essential requirements are:

- Minimal resource usage
- Cross-platform compatibility (Windows, MacOS, Linux, and eventually mobile/edge)
- Simple deployment (single executable, Docker image, or install script)
- Well-documented REST/GraphQL API with clear endpoints for all profile actions

API endpoints typically include:

- `GET /operational-identity/{user_id}` — fetch user profile
- `GET /agent-identity/{agent_id}` — fetch agent profile
- `POST /operational-identity/{user_id}` — create or update profile
- `POST /agent-identity/{agent_id}` — create or update agent profile
- `GET /log` — view change, access, and feedback logs

- `POST /backup` / `GET /backup` — manage encrypted exports/imports

A full OpenAPI/Swagger spec can be included, allowing instant plug-and-play with LLMs, IDEs, or multi-agent orchestration frameworks.

B. Database Options and File Storage

Unlike centralized databases, the Vault's storage layer is designed for **user control and flexibility**. Users can select their preferred storage method:

- **Default:** Local SQLite DB or encrypted JSON files—portable, simple, and secure.
- **Advanced:** Optionally connect to a user-owned Postgres, MySQL, or document store (for those running persistent servers or wanting enterprise features).
- **Decentralized:** Store encrypted profiles on IPFS, Arweave, or a P2P mesh, for distributed backup or multi-device access.

All data is encrypted at rest by default. Keys are stored locally, with user-chosen passwords/passphrases (never in a cloud unless the user opts in).

Storage must support:

- Fast read/write for small records (profiles, logs)
- Append-only audit logs (for rollback and forensics)
- Easy import/export of JSON for sharing, migration, or backup

---

C. Authentication, Encryption, and Access Control

The Vault is built around **user-controlled security**:

Authentication

- Local-only mode: Only the local user can access or modify the Vault (via system credentials or a local app password).
- Remote/Tunnelled mode: Token-based or OAuth-style authentication when sharing the API remotely (e.g., via ngrok or VPN).
- Optional multi-factor authentication for admin actions.

Encryption

- All operational identities and logs are encrypted at rest using strong, modern algorithms (e.g., AES-256).
- API communication is encrypted via HTTPS or SSH.
- Optional end-to-end encryption for P2P/decentralized sync (e.g., keys never leave the device).

Access Control

- Fine-grained permissioning: Users define who (or what agents) can access, edit, or query profiles.
- Permission fields in agent profiles restrict agent actions at the Vault level (e.g., an agent profile with “no file deletion” cannot make API calls that would trigger a delete).
- Audit trails record every access, change, and attempted breach.

Backup and Recovery

- Users can export encrypted snapshots of the entire Vault (or selected profiles/logs) for offsite or multi-device use.
- Recovery processes are documented and user-driven, ensuring no third party is required to regain access.

D. Future-Proof, Open, and Extensible

The Vault’s server is not a closed system. It is designed from the start to:

- Support open API specs for easy integration with new agents, platforms, or AI models.
- Allow users or devs to add custom fields, rules, or extensions to profiles—without breaking existing integrations.
- Be open source, so anyone can review, audit, and improve the codebase.

In summary:
The Vault Server Implementation is the backbone of the platform—combining privacy, portability, and power in a package users truly own and control.
It is both the "brain" and the "lockbox" of your operational identity, agent personas, and workflow memory—built for the reality of modern AI, and ready for the next wave of challenges.

---

5. User and Admin Workflows

A. First-Time Setup and Profile Creation

Setting up the Vault is designed to be fast, secure, and beginner-friendly—whether you're a solo user or deploying for a full team.
Upon installation (via one-click script, Docker, or downloadable app), the Vault walks you through a **guided onboarding**:

1. **Initialize the Vault:**
   Launch the server on your local device. The Vault checks your environment and creates a secure, encrypted data store (SQLite, JSON, or chosen DB).

2. **Create Your User Profile:**
   Using the web UI or CLI, fill out your operational identity profile. The interface prompts you—step by step—using the best-practice templates for all fields:

   - Name, user ID, tags, spirit name

    - Project context, files, goals
    - Preferred workflow, critique level, safe words, forbidden actions

3. **Create Agent Profiles:**
   For each agent or digital assistant you use, fill out an agent identity profile (role, permissions, style, capabilities, limits, logging rules, etc.).
   You can use included templates or clone/modify shared agent profiles from an internal or public library.

4. **Set Access Rules:**
   Define who (or which agents) can read or edit your profiles. Set admin passwords and, if needed, multi-factor authentication.

5. **Backup and Recovery:**
   The setup wizard prompts you to save an encrypted backup and explains how to recover your Vault if your device is lost or compromised.

Once onboarding is complete, you are in full control—**your operational identity is ready to use across all your AI workflows.**

B. Editing, Updating, and Versioning Identities

- **Edit via UI or CLI:**
  Make changes at any time—update project context, add new collaborators, tweak agent personalities, or adjust rules.
  All edits are versioned; you can review, compare, or roll back to any previous state via the interface.

- **Live Validation and Guidance:**
  The Vault UI warns you if changes create conflicts, drift, or security risks (e.g., agent profile trying to override a forbidden action).

- **Change Logs and Audit Trails:**
  Every update is recorded with a timestamp, reason, and who/what initiated it. This supports both solo “memory” and enterprise-grade compliance.

C. Importing/Exporting Profiles (JSON, API, CLI)

- **Export:**
  Instantly export any user or agent profile (or full Vault snapshot) as a JSON file—encrypted or plain, as needed.
  Export profiles to bring your operational self to new devices, teams, or to share with other Vault users.

- **Import:**
  Load a profile from file, API endpoint, or decentralized store (e.g., IPFS hash). All imports are validated before activation, preventing corrupted or malicious configs.

- **API Calls:**
  Developers can automate profile import/export using the Vault's documented API, integrating with workflow tools or deployment scripts.

D. Using the Vault with LLMs, Agents, and Apps

- **For LLMs and Chatbots:**
  Point your model to the Vault API endpoint (e.g., `http://localhost:3000/operational-identity/nic_bogaert`) at startup, or paste in the exported JSON.
  The model now adapts to your preferences, project context, and meta-rules instantly—no more prompt engineering or lost history.

- **For Agent Frameworks:**
  When spinning up a multi-agent system (e.g., LangChain, AutoGen, CrewAI), the orchestrator fetches each agent's profile from the Vault, enforcing permissions, roles, and collaboration logic as declared.

- **For Workflow Apps and IDEs:**
  Plugins or extensions can query the Vault for the current user/agent context, allowing code editors, project management, or automation tools to adapt their UI, suggested actions, or logging to the user's preferences.

- **Live Usage:**
  As you work, the Vault keeps context up-to-date:

    - Logging session feedback

- Tracking agent actions
- Alerting on drift or conflict
- Generating compliance exports

E. Team and Collaborative Workflows

- **Shared Team Vaults:**
  Teams can maintain a shared Vault instance (on a local network, self-hosted cloud, or via P2P sync), with access control for different roles (admin, contributor, agent, read-only).

- **Onboarding New Team Members:**
  New users clone the Team Vault, inheriting current project context, agent lineup, and workflow rules—no more manual re-training or lost onboarding docs.

- **Audit and Knowledge Transfer:**
  Team Vault logs serve as a living knowledge base—answering “who did what, when, and why” for retrospectives, performance reviews, or regulatory compliance.

In sum:
The Vault’s user and admin workflows are designed to make privacy, continuity, and modular AI orchestration effortless.
No matter how many tools, agents, or projects you juggle, your operational self and team context are always under your control—and always one step away from plug-and-play integration.

---

6. Agent Orchestration and Management

A. Multi-Agent Teams and Role Assignment

As AI systems mature, single agents are increasingly replaced by teams of specialized digital collaborators—each with its own strengths, focus, and limits. The Vault makes **multi-agent orchestration** reliable and auditable by giving every agent a clear operational identity and enforcing the logic of "who does what, when, and how."

- **Explicit Role Assignment:**
  Every agent is registered in the Vault with a unique identity profile, including its core role, signature style, permissions, and boundaries.
  Example:
    - "Gilligan" (critical drift-corrector, never affirms blindly, can escalate for reset)
    - "Athena" (investor dashboard, formal and data-driven, cannot override user files)
    - "Neo" (frontline agent, handles everyday tasks and friendly support)

- **Dynamic Team Formation:**
  Teams can be composed or reconfigured at will—agents are added, removed, or reassigned simply by updating their Vault profiles.
  This modularity means project pivots, staffing changes, or new agent deployment happen with zero confusion and minimal risk.

- **Cross-Project Reuse:**
  Once created, agent profiles can be exported/imported or shared as templates, making best-practice agents portable across workflows, teams, and organizations.

B. Capabilities, Limits, and Enforcement

- **Capabilities:**
  Each agent's operational identity specifies not just what it "is" but what it can *do*—from API calls to file editing, session resets, or issuing phase corrections.

- **Limits:**
  Hard boundaries are encoded ("cannot delete files", "never access internet", "must not override user decisions") and enforced both at the API level (Vault rejects forbidden actions) and within agent orchestration frameworks.

- **Enforcement:**
  Orchestration engines (LangChain, CrewAI, SuperAGI, custom stacks) integrate with the Vault to fetch, enforce, and monitor agent permissions and constraints in real time.
  Attempted breaches are logged and can trigger alerts, automated resets, or escalation

protocols.

C. Cross-Agent Collaboration, Trust Handshakes, and Verification

- **Trust Handshakes:**
  Before agents collaborate, they can perform a “trust handshake”—verifying each other’s operational identities, capabilities, and compliance with meta-rules (e.g., “critique_mandatory: true”).

- **Role Negotiation:**
  Multi-agent chains or swarms dynamically allocate tasks by reading role, capability, and permission fields, ensuring each step is handled by the most qualified and compliant agent.

- **Drift and Impersonation Detection:**
  Vault-stored profiles and logs provide tamper-evident references, so if an agent “drifts” (deviates from its declared role/personality) or is spoofed, the system can detect and correct or quarantine the problem.

D. Audit Trails and Session Logs

- **Comprehensive Logging:**
  Every agent action, context change, collaboration, and conflict is recorded in the Vault’s audit log. Logs are timestamped, versioned, and (optionally) cryptographically signed for tamper-evidence.

- **Transparency and Accountability:**
  Users and admins can trace back through any workflow or decision chain—seeing not just “what happened,” but “who (or what) did it, when, and under what rules.”

- **Forensic Analysis and Debugging:**
  When things go wrong (e.g., unexpected system drift, errors, regulatory audits), session logs and agent identity histories make it possible to reconstruct, diagnose, and fix problems—without ambiguity or blame-shifting.

- **Knowledge Sharing:**
  Teams can export anonymized logs to build “institutional memory”—a living resource for training, onboarding, and continuous improvement.

Agent orchestration is no longer a mysterious dance of bots—it’s a transparent, modular, and auditable process.
The Vault’s approach makes scaling to hundreds (or thousands) of digital collaborators as safe and manageable as hiring a new teammate—every agent always knows its role, its limits, and its responsibilities, and every action leaves a traceable record.

---

7. Integration Patterns

A. API Endpoints and Usage (REST/GraphQL)

The Vault’s real power is its ability to make identity and context instantly accessible wherever you need it.
Every profile—user or agent—can be fetched, updated, or queried using clean, well-documented API endpoints:

- **RESTful Endpoints:**
  - `GET /operational-identity/{user_id}` — Retrieve the current user operational profile.
  - `GET /agent-identity/{agent_id}` — Retrieve an agent’s operational identity.
  - `POST /operational-identity/{user_id}` — Create or update the user profile.
  - `POST /agent-identity/{agent_id}` — Create or update agent profiles.
  - `GET /log` — Retrieve audit, feedback, and action logs.

    - `POST /backup` / `GET /backup` — Manage encrypted backups or imports.

- **GraphQL (Optional):**
  Advanced deployments may support a GraphQL interface for querying multiple profiles, specific fields, or log entries in a single call.

- **Authentication:**
  By default, all endpoints are available locally (localhost), with optional token or OAuth authentication for remote, tunnelled, or team deployments.

- **CORS/Whitelisting:**
  For integration with web-based LLMs or cloud IDEs, the Vault supports CORS and IP whitelisting to ensure only trusted sources can access profiles.

B. CLI and UI Integration

- **Command-Line Interface (CLI):**
  Power users and devs can manage, update, or query profiles, logs, and agent states using simple terminal commands.
  Example:

    - `vault profile show`

    - `vault agent add --from-template reviewer.json`

    - `vault log export --since 2025-09-01`

- **Web UI:**
  For broader accessibility, a minimalist web interface allows:

    - Profile editing and field validation

    - Side-by-side comparison of user and agent profiles

    - Live viewing and searching of logs and audit trails

    - One-click import/export, backup, and team vault management

- **Automations:**
  The CLI and UI can both trigger scripts for routine tasks (nightly backups, team profile

sync, drift alerting).

C. Plug-and-Play with LLMs, Chatbots, IDEs, and Multi-Agent Frameworks

- **For LLMs and Chatbots:**
  Simply provide a pointer to your Vault's API endpoint or upload the exported JSON profile at the start of a session.
  Modern LLMs and agentic models can instantly ingest and honor your rules, preferences, and project context, minimizing manual prompt engineering.

- **For IDEs and Developer Tools:**
  Extensions or plugins query the Vault for real-time context—adapting code suggestions, file actions, and interface behaviors to your working style, preferences, and current goals.

- **For Multi-Agent Frameworks:**
  When launching orchestrated agent workflows (LangChain, AutoGen, CrewAI), each agent's operational profile is fetched and enforced—roles, permissions, and boundaries all set before runtime.

- **Low-Lift Integration:**
  Because profiles are standard JSON or API-accessible, almost any tool can integrate with just a few lines of code—using Python's `requests` library, JavaScript's `fetch`, or any HTTP client.

D. Tunneling, P2P, and Decentralized Sharing

- **Tunneling (e.g., ngrok, Tailscale):**
  Temporarily expose your Vault's local API to trusted external tools, collaborators, or remote LLM sessions. All tunnels are opt-in, encrypted, and user-controlled.

- **P2P Sync:**
  Sync profiles, logs, or full Vaults across multiple devices or team members using secure, peer-to-peer protocols—no central server needed.

- **Decentralized Storage (IPFS, Arweave, etc.):**
  For high resilience or public template sharing, encrypt and pin selected profiles or backups to a decentralized network.
  Users can “publish” agent templates or context snapshots, and teams can restore from distributed backups in seconds.

Integration Patterns are designed to make the Vault the “control tower” for digital identity, context, and trust—wherever you work, build, or collaborate.
From solo creators to global dev teams, the Vault slots in with minimal friction, giving you total control without breaking your workflow or security model.

---

8. Privacy, Security, and Compliance

A. Local-First and Self-Hosted Best Practices

The Vault is privacy-first by design:
Every workflow, storage decision, and integration assumes that the user is the sole owner and authority over their data. Profiles, logs, and operational context live on the user’s device—encrypted and inaccessible to third parties unless the user explicitly chooses to share.

- **No Default Cloud Storage:**
  The Vault never transmits, syncs, or copies your data off-device unless you opt in. No analytics, telemetry, or background “feature” that would compromise privacy.

- **Local Encryption:**
  All data (profiles, logs, audit trails) is encrypted at rest using strong, modern cryptography (AES-256 or better). Users set their own keys or passphrases, with recovery options documented at onboarding.

- **Open Source Codebase:**
  The entire Vault stack is open source, so users and teams can audit for vulnerabilities, backdoors, or privacy violations. Transparency is a core pillar of trust.

B. Encryption at Rest and in Transit

1. Encryption at Rest:

- Every operational identity, agent profile, and log entry is encrypted before writing to disk.
- Keys are stored locally (never transmitted to third parties) and can be rotated or revoked.
- Supports full-Vault encryption and selective export encryption (e.g., export a profile to share without decrypting the entire database).

2. Encryption in Transit:

- The API server only communicates via secure channels: HTTPS, SSH, or authenticated P2P tunnels.
- When sharing via decentralized storage or public backup, all exports are encrypted with recipient-chosen keys.

3. Zero-Knowledge/Selective Disclosure (Advanced):

- For teams or regulated industries, the Vault can support zero-knowledge proofs or selective disclosure, allowing you to prove the integrity of a profile or log without exposing sensitive content.

C. Regulatory Considerations (GDPR, AI Act, User Consent)

Compliance is built-in, not bolted on:
The Vault anticipates and exceeds the requirements of global privacy regulations:

- **GDPR:**
    - Data portability: Users can export their full operational identity or logs at any time, in standard, machine-readable formats.

    - Right to be forgotten: Complete Vault erasure and secure deletion tools are provided.

    - Audit logs and access records are available for review or export.

- **AI Act (EU and Emerging Global Standards):**

    - Every agent and user profile has explicit, auditable boundaries and permissions, supporting high-risk AI transparency requirements.

    - Logs and version histories provide traceable, tamper-evident records of every operational change or agent action.

- **User Consent and Ownership:**

    - No data sharing, syncing, or cloud exposure without clear, informed user consent.

    - Every API call and export/import requires user confirmation (unless specifically automated by the user for a trusted workflow).

D. Auditability and Tamper-Proof Logs

- **Every Action is Logged:**
  Every profile update, agent action, and permission change is time-stamped and added to an immutable audit trail—supporting both self-debugging and regulatory compliance.

- **Tamper-Evident Logging:**
  Optionally, logs can be cryptographically signed or chained (blockchain-style) so that any unauthorized modification is instantly detectable.

- **Log Retention and Redaction:**
  Users control how long logs are kept, and can redact or anonymize entries before export—aligning with best practices in privacy and compliance.

The Vault doesn't just promise privacy and compliance—it builds them into every layer. Whether you're a solo user protecting your creative process, a dev team worried about agent drift, or a company facing regulatory audits, the Vault's privacy-by-design DNA

means you never have to compromise.
Your data. Your rules. Your peace of mind.

---

9. Advanced Extensions

A. Decentralized/Distributed Identity (IPFS, ZKPs)

As the ecosystem matures, some users and teams will want even greater resilience, privacy, and portability.
The Vault is designed to support decentralized and distributed identity out of the box—empowering users to take their operational context anywhere, or share it across any number of devices and agents, with total sovereignty.

- **IPFS Integration:**
  Pin encrypted profile snapshots, agent templates, or full audit logs to the InterPlanetary File System (IPFS), providing permanent, tamper-proof storage that's always retrievable by hash—no centralized server required.
    - **Use case: Publishing open-source agent templates or sharing audit trails for peer verification.**

- **Zero-Knowledge Proofs (ZKPs):**
  For regulated industries, enterprise, or high-trust environments, the Vault can generate cryptographic proofs of profile integrity or rule enforcement *without* revealing private details.
    - **Example: Prove to a collaborator or regulator that your agent *never* took a forbidden action, or that your profile's meta-rules were enforced, without exposing the underlying logs.**

- **Distributed Identity (DID) Support:**
  The Vault can interoperate with emerging standards for decentralized digital identity (W3C DID), allowing users to sign, verify, and sync their operational context across platforms, organizations, or blockchains.

B. Shared Team Vaults and Collaboration

- **Multi-User Vaults:**
  Support for collaborative Vaults, where teams share operational context, agent lineups, and workflow logs—each with their own access controls and permissions.
    - **Roles: Admin, contributor, agent-only, read-only.**
    - **Team onboarding is instantaneous: clone the Team Vault, inherit all context and agent rules.**
- **Live Sync and Merge:**
  Vaults can sync peer-to-peer or through user-owned cloud instances, with built-in conflict detection and resolution (à la Git).
  Change logs and merge requests are managed just like in distributed codebases.
- **Collaborative Agent Design:**
  Teams can co-design agents, publish or import best-practice templates, and assign agents to roles as needed. Every change is logged, attributed, and versioned.

C. Hooks for Context-Aware LLMs and RAG Pipelines

- **Retrieval-Augmented Generation (RAG):**
  The Vault can provide context, session memory, and meta-rules to RAG pipelines or context-aware LLMs, ensuring answers, completions, or agent behaviors always align with the latest user/agent state.
    - **Plug the Vault into search, summarization, or code generation agents for smarter, drift-resistant outputs.**
- **Event Hooks and Automations:**
  Define triggers or webhooks for key Vault events (e.g., “profile updated”, “agent reset”, “drift detected”) that kick off automations in your workflow tools, alerting, or even Slack/Discord channels.
- **Custom Plugins and Extensions:**
  Open API and CLI support makes it easy to develop and share new Vault features:

- **Language packs for international teams**
- **Plugin marketplaces for agents or profiles**
- **Integration adapters for emerging agent frameworks**

D. Future Ecosystem Features

- **Vault Marketplace:**
  Public index of open, trusted agent templates, operational identities, and best-practice workflows—shareable via IPFS, P2P, or user-verified APIs.

- **Automated Compliance Exports:**
  One-click export of compliance or audit packs, ready for regulator review (GDPR, AI Act, SOC 2, etc.).

- **Hybrid Cloud/Edge Operation:**
  Run your Vault on a personal server, mobile device, or secure enclave, syncing only what you need and when you need it—ensuring "always-on" operational identity, even in hybrid or offline environments.

Advanced Extensions make the Vault not just a tool, but a platform—a living ecosystem ready for the next generation of AI, compliance, and user-sovereign digital collaboration. Whether you want to go fully decentralized, coordinate global teams, or automate agent management at scale, the Vault is architected for radical flexibility and future growth.

---

10. Future-Proofing and Roadmap

A. Planned Features and Scalability

The Vault is architected for a world where AI, digital agents, and human-AI teams are rapidly evolving.
Every design choice is made with adaptability, user control, and security as first principles.
Here's how the system is built to last—and where it's headed:

- **Planned Features:**
    - **Mobile & Edge Deployments:**
      Lightweight Vault instances for mobile devices, tablets, and edge hardware (e.g., Raspberry Pi), ensuring your operational identity is always at hand.
    - **Plugin Ecosystem:**
      Official and community-developed plugins for agent extensions, new integration hooks, analytics dashboards, and cross-platform connectors.
    - **Adaptive UI/UX:**
      Context-aware, personalized interfaces that adapt to the user's operational profile—making complex agent management accessible to non-technical users.
    - **Automated Drift Detection & Correction:**
      Advanced logic for detecting and remediating "agent drift" or role confusion—reducing manual troubleshooting and boosting reliability in multi-agent systems.
    - **Multi-Language & Accessibility Support:**
      Internationalization for global teams, and accessible design for neurodiverse or differently-abled users.
    - **Smart Team Sync & Conflict Resolution:**
      Git-like diff/merge tools for resolving profile conflicts, team rule negotiation, and collaborative context editing.
    - **Consent Management & AI Bill of Rights:**
      User-controlled policies for consent, data use, and transparency—enabling Vault users to meet or exceed evolving regulatory and ethical standards.
- **Scalability:**
    - **The Vault supports scaling from single-user, single-device operation up to enterprise or global multi-agent deployments.**

- **Storage, logs, and profile management are optimized for tens of thousands of agents, years of session history, and high-frequency workflows—without sacrificing speed or simplicity.**

B. Modular Upgrades and Plugin Ecosystem

- **Modular Design:**
  Every Vault component—storage, API server, UI, log engine, integration hooks—can be independently swapped, upgraded, or extended.
  This means you're never stuck waiting for a monolithic update or major version bump. Teams and developers can patch, fork, or improve the Vault as new needs emerge.

- **Plugin Marketplace:**
  Both official and community-built plugins will extend Vault functionality:

  - **New agent "starter kits" (e.g., legal assistant, critic, data analyst)**
  - **Workflow integrations for IDEs, chat platforms, CRM, and more**
  - **Compliance and audit plugins for industry-specific requirements**

C. Addressing Emerging AI Ecosystem Challenges

The AI/Web landscape will only get more fragmented, modular, and high-stakes in coming years. The Vault is designed to anticipate—and meet—those challenges:

- **Swarm & Agent Impersonation:**
  Verifiable agent profiles and trust handshakes combat the risk of agent spoofing or swarm attacks in open, decentralized agent networks.

- **Memory Blackouts & Context Rot:**
  Persistent, versioned logs and context carryover tools prevent knowledge loss, session resets, and productivity-killing "Groundhog Day" onboarding.

- **Regulatory Black Holes:**
  Tamper-evident logs, one-click compliance exports, and user-controlled consent

make audits routine—not a crisis.

- **Cognitive Fragmentation & Emotional Drift:**
  Unified, portable user profiles act as a "north star" for multi-agent teams, keeping AI behaviors aligned with true user intent across tools and time.

In short:
The Vault is more than a static product—it is a living, evolving platform designed to ride the next decade of AI transformation, always putting user sovereignty, modularity, and trust first.
Every feature, integration, and community extension adds to a foundation that's built not just for today's AI pains, but for tomorrow's unknown challenges.

---

11. Developer Guide

A. Contribution Standards

The Vault is designed as an open, community-driven platform.
To ensure long-term reliability, security, and adaptability, contributions from developers and users are encouraged—but must meet high standards of quality and transparency.

- **Open Source Repository:**
  All core code, APIs, UI components, and integration plugins are maintained in a public version-controlled repository (e.g., GitHub, GitLab).

- **Clear Contribution Guidelines:**
  The project includes a CONTRIBUTING.md document specifying how to propose features, submit pull requests, file issues, and participate in community discussions.

- **Code Review and Testing:**
  All major changes are reviewed by core maintainers and must pass automated tests for security, functionality, and backward compatibility.

- **Documentation First:**
  Every new feature, extension, or API change requires clear documentation and

usage examples before merging.

B. Custom Field/Extension Patterns

The Vault is built for extensibility.
Both user and agent operational identities can be extended with custom fields and structures, ensuring the system adapts as the AI ecosystem evolves.

- **Custom Fields:**
  Developers and power users can add domain-specific fields to any profile (e.g., industry certifications, custom agent skills, unique compliance rules).

- **Schema Validation:**
  The Vault enforces validation on both core and custom fields, preventing malformed data or drift from breaking workflows.

- **Backward Compatibility:**
  Extensions are always opt-in and versioned; old profiles continue to work even as new capabilities are added.

C. API and Template Versioning

As more platforms, agent frameworks, and regulatory requirements appear, the Vault's APIs and templates are versioned for clarity and stability.

- **API Versioning:**
  All endpoints and responses declare their version; breaking changes require a new version (e.g., /v1/, /v2/).

- **Template Versioning:**
  Both user and agent profile templates include version and last_updated fields, enabling easy rollback, audit, and migration.

- **Migration Tools:**
  The Vault ships with migration scripts to upgrade profiles and logs between major versions, minimizing disruption.

D. Open Source and Community Practices

- **Community Engagement:**
  Regular roadmaps, open discussions, and community polls ensure development aligns with user needs.

- **Bug Bounties and Security Audits:**
  Formal programs incentivize finding vulnerabilities, with all issues transparently tracked and resolved.

- **Plugin & Integration Marketplace:**
  Developers can publish and maintain plugins (for agents, workflows, integrations) in an official registry—expanding the Vault's reach and utility.

E. Example Developer Workflow

1. **Fork and Clone:**
   Start by forking the Vault repository and cloning it locally.

2. **Set Up Your Dev Environment:**
   Follow the provided quickstart (requirements, Docker, environment variables).

3. **Develop and Test:**
   Write new features, extensions, or plugins.
   Run automated tests (`npm test`, `pytest`, etc.).

4. **Document:**
   Add or update docs and usage examples.

5. **Submit Pull Request:**
   Propose your changes, describe rationale, and participate in code review.

6. **Merge and Release:**
   After review, changes are merged, versioned, and released with full changelogs.

The Developer Guide ensures that the Vault remains not only secure and reliable, but a living, user-driven platform—growing with every contribution and always aligned with the changing needs of its community.

---

12. Appendices

A. Full Blank Templates (User and Agent Profiles)

User Operational Identity Profile — Blank Template

```
{
  "user_id": "",
  "canonical_name": "",
  "spirit_name": "",
  "project_affiliations": [],
  "identity_tags": [],
  "version": "1.0.0",
  "last_updated": "",
  "project_context": {
    "current_project": "",
    "phase": "",
    "current_files": [],
    "active_collaborators": [],
    "subsystems": [],
    "goals": []
  },
  "interaction_preferences": {
```

```
  "formality": "",
  "depth": "",
  "step_by_step": false,
  "beginner_friendly": false,
  "plain_language": false,
  "no_boxes": false,
  "pushback": false,
  "critique_mandatory": false,
  "confirmation_required": false,
  "formatting_rules": []
},
"meta_rules": {
  "recursive_feedback": false,
  "drift_tracking": false,
  "contradiction_alerts": false,
  "require_honest_pushback": false,
  "preserve_session_memory": false,
  "require_context_carryover": false,
  "log_all_exchanges": false,
  "always_explain_decisions": false,
  "safe_words": [],
  "forbidden_actions": []
},
"session_state": {
```

```
  "phase": "",
  "waiting_for": "",
  "last_feedback": "",
  "last_action": "",
  "timestamp": ""
 }
}
```

Agent Operational Identity Profile — Blank Template

```
{
  "agent_id": "",
  "canonical_name": "",
  "version": "1.0.0",
  "last_updated": "",
  "role": "",
  "symbolic_signature": [],
  "description": "",
  "capabilities": [],
  "limitations": [],
  "persona": "",
  "voice_style": "",
  "quotes_or_character_inspiration": [],
```

```
  "special_style_notes": [],
  "permissions": [],
  "authority": [],
  "enforcement_rules": [],
  "forbidden_actions": [],
  "session_state": "",
  "last_action": "",
  "last_feedback": "",
  "log_fields": [],
  "timestamp": ""
}
```

B. Example Profiles and Real-World Use Cases

Example User Operational Identity Profile

{

"user_id": "riley_dev",

"canonical_name": "Riley M. Thompson",

"spirit_name": "Skybuilder",

"project_affiliations": ["OpenAgentHub", "EchoSync"],

"identity_tags": ["architect", "AI evangelist", "privacy-first"],

"version": "1.1.2",

```
"last_updated": "2025-09-13T14:30:00Z",
"project_context": {
  "current_project": "EchoSync V2 Launch",
  "phase": "Testing & Rollout",
  "current_files": ["spec_v2.pdf", "feedback.log"],
  "active_collaborators": ["Jess Yu", "Emil Varga"],
  "subsystems": ["Sync Engine", "Profile Loader"],
  "goals": ["Complete QA", "Draft marketing site"]
},
"interaction_preferences": {
  "formality": "friendly",
  "depth": "maximum",
  "step_by_step": true,
  "beginner_friendly": false,
  "plain_language": true,
  "no_boxes": true,
  "pushback": true,
  "critique_mandatory": false,
  "confirmation_required": true,
  "formatting_rules": ["avoid tables", "explain technical terms"]
},
"meta_rules": {
  "recursive_feedback": true,
  "drift_tracking": false,
```

```
    "contradiction_alerts": true,
    "require_honest_pushback": true,
    "preserve_session_memory": true,
    "require_context_carryover": false,
    "log_all_exchanges": true,
    "always_explain_decisions": true,
    "safe_words": ["timeout", "stepback"],
    "forbidden_actions": ["skip critical errors"]
  },
  "session_state": {
    "phase": "Q&A review",
    "waiting_for": "",
    "last_feedback": "",
    "last_action": "Reviewed feedback.log.",
    "timestamp": "2025-09-13T14:31:00Z"
  }
}
```

Example Agent Operational Identity Profile


```
{
  "agent_id": "echo_reviewer",
  "canonical_name": "Echo Reviewer",
  "version": "2.0.0",
```

"last_updated": "2025-09-13T14:32:00Z",

"role": "Automated Feedback Aggregator",

"symbolic_signature": ["concise", "thorough", "neutral"],

"description": "Echo Reviewer collects, summarizes, and flags key feedback from all user and agent logs, ensuring unbiased aggregation.",

"capabilities": ["summarize logs", "flag inconsistencies", "send notifications"],

"limitations": ["cannot edit user files", "cannot override user feedback", "no access to private user notes"],

"persona": "analytical but approachable",

"voice_style": "clear and unembellished",

"quotes_or_character_inspiration": ["Inspired by Google's BERT, but much more transparent."],

"special_style_notes": ["never assigns blame", "summaries capped at 300 words"],

"permissions": ["log access", "notification API"],

"authority": ["can flag feedback for review"],

"enforcement_rules": ["never change logs", "always tag flagged entries"],

"forbidden_actions": ["delete feedback", "respond to user DMs directly"],

"session_state": "active",

"last_action": "Flagged duplicate feedback.",

"last_feedback": "All logs processed without conflict.",

"log_fields": ["summary reports", "flagged entries"],

"timestamp": "2025-09-13T14:33:00Z"

}

Real-World Use Cases

- **Cross-Platform Consistency:**
  Riley's user profile lets them bring their AI workflow and preferences to any LLM or agent, instantly—no matter which platform or stack.

- **Agent Swapping and Safety:**
  "Echo Reviewer" can be deployed, swapped, or retired across projects, always with explicit permissions and enforcement of forbidden actions.

- **Team Onboarding:**
  When Riley adds a new collaborator, they export their user profile and agent templates, letting the new teammate start with the same context and safety rules—no manual onboarding or lost memory.

- **Regulatory Audit:**
  During a compliance review, all logs, actions, and profile changes are exported, demonstrating clear boundaries, permissions, and auditability for both users and agents.

C. Code Snippets and Sample Integrations

---

1. Python: Loading a User Profile from the Vault API

```
import requests


VAULT_URL = "http://localhost:3000/operational-identity/riley_dev"

response = requests.get(VAULT_URL)

profile = response.json()


print("Current Project:", profile["project_context"]["current_project"])

print("Interaction Preferences:", profile["interaction_preferences"])
```

---

2. Node.js: Fetching an Agent Profile

```
const fetch = require('node-fetch');


const VAULT_URL = "http://localhost:3000/agent-identity/echo_reviewer";


fetch(VAULT_URL)
  .then(res => res.json())
  .then(agentProfile => {
    console.log("Agent Role:", agentProfile.role);
    console.log("Permissions:", agentProfile.permissions);
  });
```

---

3. CLI Command: Exporting Logs for Compliance

```
vault log export --since 2025-09-01 --output compliance_log.json
```

---

4. Integration Example: Inject Profile into LLM Prompt

For a new session, paste the relevant JSON block or call the Vault API to load operational identity as part of the system prompt/context:

System:

Load and follow these user operational identity settings:

[Paste exported JSON from the Vault or use API endpoint]

---

5. Automating Profile Sync (Pseudocode)

```
def sync_profile(local_vault_url, remote_vault_url, profile_id):
    local = requests.get(f"{local_vault_url}/operational-identity/{profile_id}").json()
    requests.post(f"{remote_vault_url}/operational-identity/{profile_id}", json=local)
    print("Profile synced successfully!")

# Usage:
# sync_profile("http://localhost:3000", "https://myserver:3000", "riley_dev")
```

D. Troubleshooting and FAQ

Troubleshooting Common Issues

1. The Vault server won't start or is unreachable.

- **Check that your local server is running (`npm start`, `python vault.py`, etc.).**
- **Confirm you're using the correct port (default is 3000).**
- **Make sure no other application is using the same port.**
- **If accessing remotely, verify that tunneling (ngrok, Tailscale) is active and you're using the correct public URL.**

2. Can't access or update profiles.

- **Ensure the user or agent ID exists in the Vault database.**
- **Check authentication or permissions (local access vs. remote/token-protected).**
- **Look for typos in endpoint URLs or request bodies.**

3. Profiles are not saving or loading as expected.

- **Check for file system permission errors (especially on Windows/Linux).**
- **Validate JSON before importing or updating—malformed files will be rejected.**
- **Review logs using the CLI or web UI (`vault log show`).**

4. Agent actions are being blocked unexpectedly.

- **Review the agent's operational identity for forbidden actions or insufficient permissions.**
- **Look for enforcement rules that might be rejecting the action at the API level.**

5. Sync or backup fails.

- **For local backups, confirm write permissions and sufficient disk space.**
- **For decentralized sync (IPFS, P2P), ensure all required services and dependencies are running.**
- **Check for network connectivity issues or firewall blocks.**

Frequently Asked Questions (FAQ)

Q: Can I run the Vault on any device?
A: Yes! The Vault is designed for desktops (Windows, MacOS, Linux), servers, and (with lightweight versions) mobile and edge devices like Raspberry Pi.

Q: What happens if I lose my encryption key or passphrase?
A: For security, the Vault cannot recover encrypted data without your key. Always back up your keys in a secure location—preferably offline.

Q: Is it safe to use the Vault for compliance-bound industries?
A: Yes. The Vault's audit logs, encryption, access controls, and export tools are designed with GDPR, AI Act, and similar regulations in mind. Regular security audits and open source transparency further support compliance.

Q: How do I share my profile or agent with a team?
A: Export your profile as encrypted JSON, or use Vault's team sync or decentralized sharing features (e.g., P2P, IPFS). Always control who can import or access your data.

Q: Can I use the Vault with any AI or agent platform?
A: Absolutely. The Vault's open API and standard JSON formats are designed for integration with any LLM, agent framework, IDE, or workflow tool—now and in the future.

Q: What if I want to add custom fields or use plugins?
A: The Vault is fully extensible—custom fields, plugins, and community extensions are encouraged. Just follow schema validation and contribution guidelines to maintain interoperability.

Q: Who owns my data?
A: You do—always. The Vault is self-hosted and open source; you never surrender your operational identity to a vendor or cloud provider.

E. Glossary of Terms

Agent Operational Identity:
A structured profile that defines the role, capabilities, boundaries, and personality of an AI agent or digital assistant within the Vault system.

Audit Log:
A tamper-evident record of every significant action, update, or access event in the Vault—supporting compliance, debugging, and trust.

Capabilities:
The explicit functions or actions an agent is allowed and able to perform (e.g., summarizing, resetting sessions, file editing).

Context Carryover:
The ability to preserve and transfer operational context (project state, preferences, meta-rules) across different tools, sessions, or devices.

Decentralized Storage:
A method of storing data on distributed networks (like IPFS or Arweave), removing reliance on centralized servers and enhancing resilience and privacy.

Drift Detection:
Monitoring for deviations from an agent's intended role, permissions, or personality—used to correct errors or prevent unauthorized actions.

Encryption at Rest:
Protecting all stored data (profiles, logs, context) with cryptography, so it cannot be read if the storage device is lost or stolen.

Enforcement Rules:
Specific policies or logic blocks that determine how agent capabilities and limitations are enforced in real time by the Vault or orchestration layer.

Forbidden Actions:
Non-negotiable actions that an agent (or user) must never perform, defined at the profile level and enforced by the Vault.

Identity Vault:
The self-hosted, user-controlled platform for storing, managing, and sharing user and agent operational identities, logs, and context.

Meta-Rules:
High-level feedback, safety, and behavioral policies—such as requiring honest pushback, session memory preservation, or contradiction alerts.

Operational Identity:
A portable, structured digital profile representing a user's or agent's preferences, workflow, rules, and context—central to Vault operations.

Permissions:
Defined rights that allow agents or users to access or manipulate specific resources, data, or system functions.

Profile Versioning:
Tracking and managing changes to user or agent profiles, ensuring auditability, rollback, and schema compatibility over time.

Session State:
The live "status" of a user or agent profile—current phase, last action, feedback, and timestamp—supporting workflow continuity.

Tamper-Evidence:
Design features (e.g., cryptographic signatures, immutable logs) that make unauthorized modifications easy to detect.

User Operational Identity:
A structured profile encoding a user's working style, context, preferences, rules, and project state—used to personalize and protect all AI interactions.

Vault API:
The programmable interface (REST, GraphQL, etc.) for interacting with the Vault—enabling profile retrieval, updates, logs, and integrations with external tools.

Tab 2

```
identity-vault/
├── package.json              # Dependencies, scripts (start, dev, test)
├── .env.example              # Template for env vars (keys, ports, IPFS node)
├── .gitignore                # Ignore node_modules, .env, vault.db
├── README.md                 # Setup, usage, API docs, contribution guide
├── server.js                 # Main entry point: Express app setup, middleware, route mounting
├── db.js                     # SQLite DB init, schema creation (profiles table with JSON col),
queries (CRUD with versioning)
├── schemas.js                # JSON templates/objects for user/agent profiles (full fields from
docs, validators)
├── routes/
│   ├── profiles.js           # API routes: GET/POST/PATCH/DELETE /profiles/{type}/{id}
(user/agent), versioning
│   ├── feedback.js           # POST /feedback, GET /state (logs, drift alerts, contradictions)
│   └── agents.js             # Multi-agent specific: POST /orchestrate (team roles, handshakes),
GET /audit
├── utils/
│   ├── prompts.js            # Step-by-step CLI prompts for profile creation (groups from docs,
confirmation waits)
│   ├── encryption.js         # Crypto utils: AES encrypt/decrypt profiles (at rest/transit), key
gen from env
│   ├── ipfs.js               # IPFS pinning/export for decentralized sharing (js-ipfs lib),
hash-based queries
│   ├── zkps.js               # Stub ZKP for trust proofs (circom/wasm sim; selective disclosure of
fields)
│   ├── drift.js              # Logic: Track conceptual drift, contradiction alerts, recursive feedback
loops
│   └── integrations.js       # Hooks: LangChain memory loader, AutoGen/CrewAI agent
config, RAG filters
├── cli.js                    # CLI entry: npm run cli -- create-profile, import, export, tunnel (ngrok)
├── templates/                # Blank JSONs (auto-generated from schemas.js)
│   ├── user-template.json    # Full blank user profile
│   └── agent-template.json   # Full blank agent profile
├── plugins/                  # Modular extensions (future-proof)
│   └── example-hook.js       # Sample webhook/event for custom automations
├── tests/                    # Basic tests (using jest if added)
│   └── server.test.js        # Unit tests for routes/DB
└── vault.db                  # Generated SQLite file (gitignore'd; stores encrypted JSON blobs)
```

```
// server.js - Main Entry Point for Self-Hosted Identity Vault
// Sovereign, Portable, Privacy-First Operational Identity Server
```

```
// Version: 1.0 | Compiled: Nic Bogaert & AI.Web Systems Group, 2025
// Features: Full API for user/agent profiles, versioning, encryption, drift tracking,
// multi-agent orchestration, IPFS/ZKP integration, recursive feedback, audit logs,
// regulatory exports, LangChain/AutoGen hooks, safe word enforcement, session memory.
// Run: npm start (listens on port from .env or 3000)

require('dotenv').config(); // Load env vars (e.g., PORT=3000, ENCRYPTION_KEY=...,
IPFS_NODE=...)
const express = require('express');
const cors = require('cors'); // For cross-origin (e.g., LLM apps)
const bodyParser = require('body-parser'); // Parse JSON bodies
const sqlite3 = require('sqlite3').verbose(); // DB driver
const path = require('path');
const { encrypt, decrypt } = require('./utils/encryption'); // Privacy: Encrypt profiles
const { initDB, getProfile, saveProfile, updateProfile, logFeedback, getState } = require('./db'); //
DB ops
const userSchema = require('./schemas').user; // Templates/validators
const agentSchema = require('./schemas').agent;
const { createProfilePrompts } = require('./utils/prompts'); // CLI-like prompts (used in API too)
const { pinToIPFS, fetchFromIPFS } = require('./utils/ipfs'); // Decentralized sharing
const { proveZKP, verifyZKP } = require('./utils/zkps'); // Advanced: Trust handshakes
const { detectDrift, alertContradictions, recursiveFeedback } = require('./utils/drift'); // Meta-rules
enforcement
const { langchainLoader, autogenHook, ragFilter } = require('./utils/integrations'); // Ecosystem
hooks
const profileRoutes = require('./routes/profiles'); // User/Agent CRUD
const feedbackRoutes = require('./routes/feedback'); // Logs/state
const agentRoutes = require('./routes/agents'); // Orchestration/audit

const app = express();
const PORT = process.env.PORT || 3000;
const dbPath = path.join(__dirname, 'vault.db');

// Global middleware
app.use(cors({ origin: '*' })); // Allow LLM/localhost access; tighten for prod
app.use(bodyParser.json({ limit: '10mb' })); // Handle large JSON profiles
app.use(bodyParser.urlencoded({ extended: true }));

// Auth stub: Basic token check (expand to JWT/OAuth for teams)
app.use((req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (token !== process.env.API_TOKEN) { // Env var for simple auth
    return res.status(401).json({ error: 'Unauthorized: Invalid token' });
  }
```

```
  next();
});

// Error handling middleware (logs drifts/contradictions)
app.use((err, req, res, next) => {
  console.error('Server Error:', err); // Audit trail
  if (err.drift) detectDrift(req.body, res); // Enforce meta-rules
  res.status(500).json({ error: 'Internal error; check logs' });
});

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'OK', version: '1.0', timestamp: new Date().toISOString() });
});

// Profile creation endpoint (uses prompts for guided setup)
app.post('/profiles/create/:type', async (req, res) => { // type: 'user' or 'agent'
  try {
    const { type } = req.params;
    const schema = type === 'user' ? userSchema : agentSchema;
    const answers = await createProfilePrompts(schema, req.body.initialAnswers || {}); //
Interactive if CLI, else batch
    const profile = { ...schema.template, ...answers, version: '1.0.0', last_updated: new
Date().toISOString() };
    const encrypted = encrypt(JSON.stringify(profile), process.env.ENCRYPTION_KEY);
    const id = await saveProfile(type, encrypted, profile.user_id || profile.agent_id);
    // Optional IPFS pin for portability
    if (req.body.pinToIPFS) {
      const hash = await pinToIPFS(encrypted);
      profile.ipfs_hash = hash;
    }
    res.json({ id, profile: decrypt(encrypted, process.env.ENCRYPTION_KEY), message: 'Profile
created' });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// Load profile (with ZKP verification for trust)
app.get('/profiles/:type/:id', async (req, res) => {
  try {
    const { type, id } = req.params;
    let encrypted = await getProfile(type, id);
    if (!encrypted) return res.status(404).json({ error: 'Profile not found' });
```

```
    const profile = JSON.parse(decrypt(encrypted, process.env.ENCRYPTION_KEY));
    // ZKP for selective disclosure (advanced: prove fields without revealing all)
    if (req.query.zkpProof) {
      const proof = await proveZKP(profile, req.query.fields.split(','));
      if (!verifyZKP(proof)) return res.status(403).json({ error: 'ZKP verification failed' });
      profile.selective = proof; // Return proof only
    }
    // IPFS fallback if local missing
    if (!encrypted && profile.ipfs_hash) encrypted = await fetchFromIPFS(profile.ipfs_hash);
    // Enforce meta-rules (e.g., safe words, context carryover)
    if (profile.meta_rules.safe_words && req.query.safe_word) {
      return res.json({ action: 'halted', reason: 'Safe word triggered' });
    }
    res.json({ profile, session_state: await getState(id) }); // Preserve memory
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Update profile (partial PATCH, versioning)
app.patch('/profiles/:type/:id', async (req, res) => {
  try {
    const { type, id } = req.params;
    const updates = req.body;
    let encrypted = await getProfile(type, id);
    const profile = JSON.parse(decrypt(encrypted, process.env.ENCRYPTION_KEY));
    const newVersion = `${parseFloat(profile.version) + 0.1}`.substring(0, 5); // Semantic
versioning stub
    const updated = { ...profile, ...updates, version: newVersion, last_updated: new
Date().toISOString() };
    // Drift check before save
    const driftAlert = detectDrift(profile, updated);
    if (driftAlert) {
      await logFeedback(id, { type: 'drift', alert: driftAlert });
      if (updated.meta_rules.drift_tracking) res.json({ warning: driftAlert, profile: updated });
    }
    // Contradiction alert
    const contradiction = alertContradictions(updated);
    if (contradiction) await recursiveFeedback(updated, contradiction);
    const newEncrypted = encrypt(JSON.stringify(updated), process.env.ENCRYPTION_KEY);
    await updateProfile(type, id, newEncrypted);
    res.json({ id, version: newVersion, changes: Object.keys(updates) });
  } catch (err) {
    res.status(400).json({ error: err.message });
```

```
  }
});

// Mount sub-routes
app.use('/profiles', profileRoutes); // Extends CRUD
app.use('/feedback', feedbackRoutes); // Logs, state
app.use('/agents', agentRoutes); // Orchestration, audits

// Advanced: Integration hooks (e.g., LangChain loader)
app.post('/integrations/langchain', (req, res) => {
  const { profileId, type } = req.body;
  res.json({ loader: langchainLoader(profileId, type) }); // Returns memory module
});

// AutoGen/CrewAI hook
app.post('/integrations/autogen', (req, res) => {
  const { agentId } = req.body;
  res.json({ config: autogenHook(agentId) }); // Agent role/perms
});

// RAG filter
app.post('/integrations/rag', (req, res) => {
  const { query, profileId } = req.body;
  res.json({ filtered: ragFilter(query, profileId) }); // Tags-based
});

// Regulatory export (GDPR/AI Act compliant)
app.get('/export/regulatory/:id', async (req, res) => {
  const { id } = req.params;
  const profile = await getProfile('user', id); // Or agent
  const logs = await getState(id); // Full audit trail
  res.set('Content-Type', 'application/json');
  res.download(path.join(__dirname, `export-${id}.json`)); // Tamper-proof with timestamps
  // Auto-log export
  await logFeedback(id, { type: 'export', fields: ['all'] });
});

// Webhook for events (e.g., milestone updates)
app.post('/hooks/event', async (req, res) => {
  const { event, profileId } = req.body; // e.g., {event: 'milestone', profileId: 'nic'}
  // Trigger update reminder or automation
  console.log(`Event: ${event} for ${profileId}`);
  res.json({ acknowledged: true });
});
```

```
// Graceful shutdown
process.on('SIGINT', async () => {
  console.log('Shutting down Vault...');
  db.close((err) => {
    if (err) console.error(err.message);
    console.log('DB closed.');
  });
  process.exit(0);
});

// Init DB on start
initDB(dbPath);

// Start server
app.listen(PORT, () => {
  console.log(`Identity Vault running at http://localhost:${PORT}`);
  console.log('Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.');
  console.log('Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.');
});

module.exports = app; // For testing
```

```
// db.js - SQLite Database Layer for Self-Hosted Identity Vault
// Handles init, schema (profiles table with JSON blob for flexibility), CRUD ops for user/agent profiles,
// versioning (via timestamp/version field), logging (feedback/state tables for audits, drifts),
// encryption passthrough (store encrypted blobs), multi-agent session state, regulatory exports.
// Integrates: Query hooks for IPFS hashes, ZKP metadata, recursive feedback storage.
// Version: 1.0 | 2025 | Privacy: All data encrypted; local-only.
// Exports: initDB, getProfile, saveProfile, updateProfile, deleteProfile, logFeedback, getState, getAuditTrail.

const sqlite3 = require('sqlite3').verbose();
const path = require('path');
const { encrypt, decrypt } = require('../utils/encryption'); // Passthrough for at-rest encryption

let db; // Global DB instance (single connection for safety)

// Profiles table schema: Flexible JSON storage for user/agent (type discriminator), with versioning/audit fields
const createSchema = `
  CREATE TABLE IF NOT EXISTS profiles (
```

```
  id TEXT PRIMARY KEY,  -- user_id or agent_id
  type TEXT NOT NULL,   -- 'user' or 'agent'
  data TEXT NOT NULL,   -- Encrypted JSON blob
  version TEXT NOT NULL DEFAULT '1.0.0',
  last_updated TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ipfs_hash TEXT,       -- For decentralized pinning
  zkp_metadata TEXT     -- Proof stubs for trust handshakes
);

CREATE TABLE IF NOT EXISTS feedback_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  profile_id TEXT NOT NULL,
  type TEXT NOT NULL,   -- 'feedback', 'drift', 'contradiction', 'export'
  data TEXT NOT NULL,   -- JSON: {alert: '...', timestamp: '...'}
  timestamp TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (profile_id) REFERENCES profiles (id)
);

CREATE TABLE IF NOT EXISTS session_state (
  id TEXT PRIMARY KEY,  -- profile_id
  phase TEXT,
  waiting_for TEXT,
  last_feedback TEXT,
  last_action TEXT,
  timestamp TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id) REFERENCES profiles (id)
);

CREATE INDEX IF NOT EXISTS idx_profiles_type ON profiles(type);
CREATE INDEX IF NOT EXISTS idx_logs_profile ON feedback_logs(profile_id);
`;

// Init DB: Safe, local file-based (vault.db in project root, gitignore'd)
function initDB(dbPath) {
  return new Promise((resolve, reject) => {
    db = new sqlite3.Database(dbPath, (err) => {
      if (err) {
        console.error('DB Init Error:', err.message);
        reject(err);
      } else {
        db.exec(createSchema, (err) => {
          if (err) reject(err);
          else {
            console.log('DB Initialized: Profiles, logs, and state tables ready.');
```

```
            resolve(db);
          }
        });
      }
    });
  });
}

// Save new profile: Encrypt, insert with version/timestamp, optional IPFS/ZKP prep
async function saveProfile(type, encryptedData, id, ipfsHash = null, zkpMeta = null) {
  return new Promise((resolve, reject) => {
    const stmt = db.prepare(`
      INSERT INTO profiles (id, type, data, version, ipfs_hash, zkp_metadata)
      VALUES (?, ?, ?, ?, ?, ?)
    `);
    stmt.run(id, type, encryptedData, '1.0.0', ipfsHash, zkpMeta, function(err) {
      if (err) reject(err);
      else {
        // Init session state
        db.run(`INSERT OR IGNORE INTO session_state (id) VALUES (?)`, [id]);
        resolve(this.lastID);
      }
      stmt.finalize();
    });
  });
}

// Get profile: Fetch, decrypt, include state/logs if requested
async function getProfile(type, id, includeState = true, includeLogs = false) {
  return new Promise((resolve, reject) => {
    db.get(`
      SELECT * FROM profiles WHERE type = ? AND id = ?
    `, [type, id], (err, row) => {
      if (err) reject(err);
      else if (!row) resolve(null); // Not found
      else {
        const decrypted = decrypt(row.data, process.env.ENCRYPTION_KEY);
        if (!decrypted) reject(new Error('Decryption failed: Invalid key'));
        const profile = JSON.parse(decrypted);
        if (includeState) {
          db.get(`SELECT * FROM session_state WHERE id = ?`, [id], (err, state) => {
            if (!err) profile.session_state = state;
            if (includeLogs) getAuditTrail(id).then(logs => profile.logs = logs);
            resolve({ ...row, data: profile });
```

```
        });
      } else {
        resolve({ ...row, data: profile });
      }
    }
  });
 });
}

// Update profile: Version bump, encrypt updates, log changes for audit
async function updateProfile(type, id, updates, partial = true) {
 return new Promise((resolve, reject) => {
  // Fetch current for versioning
  db.get(`SELECT data, version FROM profiles WHERE type = ? AND id = ?`, [type, id], async
(err, row) => {
    if (err || !row) reject(new Error('Profile not found'));
    else {
      const current = JSON.parse(decrypt(row.data, process.env.ENCRYPTION_KEY));
      const updatedProfile = partial ? { ...current, ...updates } : updates;
      const newVersion = `${parseFloat(row.version) + 0.1}`.substring(0, 5);
      updatedProfile.version = newVersion;
      updatedProfile.last_updated = new Date().toISOString();
      const encrypted = encrypt(JSON.stringify(updatedProfile),
process.env.ENCRYPTION_KEY);

      // Update DB
      db.run(`
        UPDATE profiles SET data = ?, version = ?, last_updated = CURRENT_TIMESTAMP
        WHERE type = ? AND id = ?
      `, [encrypted, newVersion, type, id], function(err) {
        if (err) reject(err);
        else {
          // Log change for audit
          logFeedback(id, { type: 'update', changes: Object.keys(updates || {}), oldVersion:
row.version });
          // Update state timestamp
          db.run(`UPDATE session_state SET timestamp = CURRENT_TIMESTAMP WHERE id
= ?`, [id]);
          resolve(this.changes);
        }
      });
    }
  });
 });
```

```
}

// Delete profile: Soft-delete via flag (for audits); hard-delete optional
async function deleteProfile(type, id, hard = false) {
  return new Promise((resolve, reject) => {
    if (hard) {
      db.run(`DELETE FROM profiles WHERE type = ? AND id = ?`, [type, id], function(err) {
        if (err) reject(err);
        else {
          // Cascade delete logs/state
          db.run(`DELETE FROM feedback_logs WHERE profile_id = ?`, [id]);
          db.run(`DELETE FROM session_state WHERE id = ?`, [id]);
          resolve(this.changes);
        }
      });
    } else {
      // Soft: Add deleted flag to data
      updateProfile(type, id, { deleted: true, version: '0.0.0' }).then(resolve).catch(reject);
    }
  });
}

// Log feedback/drift/contradictions: For recursive loops, audits, regulatory
async function logFeedback(profileId, logData) {
  return new Promise((resolve, reject) => {
    const stmt = db.prepare(`
      INSERT INTO feedback_logs (profile_id, type, data)
      VALUES (?, ?, ?)
    `);
    logData.timestamp = new Date().toISOString();
    stmt.run(profileId, logData.type, JSON.stringify(logData), function(err) {
      if (err) reject(err);
      else resolve(this.lastID);
      stmt.finalize();
    });
  });
}

// Get state: Session phase, waiting, last action/feedback (memory preservation)
async function getState(id) {
  return new Promise((resolve, reject) => {
    db.get(`SELECT * FROM session_state WHERE id = ?`, [id], (err, row) => {
      if (err) reject(err);
      else resolve(row || { id, phase: '', waiting_for: '', last_feedback: '', last_action: '' });
```

```
    });
  });
}

// Update state: For carryover, e.g., after agent actions
async function updateState(id, updates) {
  return new Promise((resolve, reject) => {
    const fields = Object.keys(updates).map(k => `${k} = ?`).join(', ');
    const values = Object.values(updates);
    values.push(id, ...values); // id first, then updates
    db.run(`
      UPDATE session_state SET ${fields}, timestamp = CURRENT_TIMESTAMP
      WHERE id = ?
    `, values, function(err) {
      if (err) reject(err);
      else resolve(this.changes);
    });
  });
}

// Get audit trail: Full logs for profile (regulatory exports, drift review)
async function getAuditTrail(profileId, limit = 100) {
  return new Promise((resolve, reject) => {
    db.all(`
      SELECT * FROM feedback_logs
      WHERE profile_id = ?
      ORDER BY timestamp DESC
      LIMIT ?
    `, [profileId, limit], (err, rows) => {
      if (err) reject(err);
      else {
        rows.forEach(row => {
          row.data = JSON.parse(row.data);
        });
        resolve(rows);
      }
    });
  });
}

// Close DB (graceful shutdown)
function closeDB() {
  if (db) {
    db.close((err) => {
```

```
      if (err) console.error('DB Close Error:', err.message);
      else console.log('DB closed safely.');
    });
  }
}

// Export for server.js
module.exports = {
  initDB,
  saveProfile,
  getProfile,
  updateProfile,
  deleteProfile,
  logFeedback,
  getState,
  updateState,
  getAuditTrail,
  closeDB
};
```

```
// schemas.js - JSON Schemas and Templates for User/Agent Operational Identities
// Based on Operational Identity API docs (2025): Full fields for user/agent profiles,
// including project_context, interaction_preferences, meta_rules, session_state.
// Features: Templates (blank objects), validators (simple schema checks for required
fields/types),
// defaults (e.g., version '1.0.0', booleans false), example populator for testing.
// Advanced: Semantic validation (e.g., arrays must be non-empty if required, safe_words
unique),
// integration hooks (e.g., derive tags from affiliations), export for prompts/db/routes.
// Usage: const { user, agent } = require('./schemas'); user.template; user.validate(profile);
// Version: 1.0 | Privacy: No PII in schemas; validate before encrypt.

const USER_TEMPLATE = {
  user_id: '', // Required: Unique handle (e.g., 'nic_bogaert')
  canonical_name: '', // Required: Full legal/professional name
  spirit_name: '', // Optional: Nickname/alter ego
  project_affiliations: [], // Array: e.g., ['ProjectX', 'Team Alpha']
  identity_tags: [], // Array: e.g., ['creative_coder', 'analytical_thinker']
  version: '1.0.0', // Auto: Semantic versioning
  last_updated: '', // Auto: ISO timestamp
  project_context: {
    current_project: '', // Required: e.g., 'AI Vault MVP'
    phase: '', // e.g., 'ideation', 'development'
```

```
    current_files: [], // Array: e.g., ['requirements.docx', 'main.py']
    active_collaborators: [], // Array: e.g., ['Alice Smith', 'Bob Lee']
    subsystems: [], // Array: e.g., ['API Gateway', 'UI/UX']
    goals: [] // Array: e.g., ['Finish MVP', 'Launch beta']
  },
  interaction_preferences: {
    formality: '', // e.g., 'stoic', 'casual', 'technical'
    depth: '', // e.g., 'maximum', 'summary', 'minimal'
    step_by_step: false, // Boolean
    beginner_friendly: false, // Boolean
    plain_language: false, // Boolean: Avoid jargon
    no_boxes: false, // Boolean: No tables/markdown
    pushback: false, // Boolean: Challenge ideas
    critique_mandatory: false, // Boolean: Always honest feedback
    confirmation_required: false, // Boolean: Wait after steps
    formatting_rules: [] // Array: e.g., ['always start with summary', 'no code blocks']
  },
  meta_rules: {
    recursive_feedback: false, // Boolean: Loops for improvement
    drift_tracking: false, // Boolean: Alert conceptual shifts
    contradiction_alerts: false, // Boolean: Flag inconsistencies
    require_honest_pushback: false, // Boolean: No simulated agreement
    preserve_session_memory: false, // Boolean: Carry context
    require_context_carryover: false, // Boolean: Enforce in chains
    log_all_exchanges: false, // Boolean: Audit everything
    always_explain_decisions: false, // Boolean: Reasoning transparency
    safe_words: [], // Array: e.g., ['pause', 'stop'] - unique strings
    forbidden_actions: [] // Array: e.g., ['skip steps', 'summarize early']
  },
  session_state: {
    phase: '', // e.g., 'waiting_review'
    waiting_for: '', // e.g., 'feedback'
    last_feedback: '', // String: Last log entry
    last_action: '', // e.g., 'updated goals'
    timestamp: '' // Auto: ISO
  }
};

const AGENT_TEMPLATE = {
  agent_id: '', // Required: Unique ID (e.g., 'coder_agent_v1')
  name: '', // Required: Display name
  core_role: '', // Required: e.g., 'code reviewer', 'dashboard assistant'
  description: '', // Short: Purpose/function
  strengths: [], // Array: e.g., ['debugging', 'UI design']
```

```
  capabilities: [], // Array: What it can do, e.g., ['access API', 'generate code']
  limitations: [], // Array: What it can't, e.g., ['no file writes', 'no external calls']
  tone: '', // e.g., 'formal', 'playful', 'irrelevant'
  permissions: [], // Array: e.g., ['read_files', 'log_actions']
  enforcement_rules: [], // Array: Meta e.g., ['never overwrite', 'log drift']
  version: '1.0.0',
  last_updated: '',
  ipfs_hash: '', // For pinning
  zkp_metadata: {} // Stub: { fields: ['role'], proof: null }
  // Note: Agents can embed user-like sections if hybrid (e.g., interaction_preferences)
};

const EXAMPLE_USER = {
  ...USER_TEMPLATE,
  user_id: 'nic_bogaert',
  canonical_name: 'Nic Bogaert',
  spirit_name: 'ProtoForge',
  project_affiliations: ['Identity Vault', 'AI.Web Systems'],
  identity_tags: ['visionary_leader', 'creative_coder'],
  last_updated: '2025-09-14T00:00:00Z',
  project_context: {
    current_project: 'Self-Hosted Vault MVP',
    phase: 'development',
    current_files: ['server.js', 'db.js'],
    active_collaborators: ['Grok AI'],
    subsystems: ['API Layer', 'Crypto Utils'],
    goals: ['Implement full CRUD', 'Add IPFS']
  },
  interaction_preferences: {
    formality: 'technical',
    depth: 'maximum',
    step_by_step: true,
    beginner_friendly: false,
    plain_language: true,
    no_boxes: true,
    pushback: true,
    critique_mandatory: true,
    confirmation_required: true,
    formatting_rules: ['explain decisions', 'no fluff']
  },
  meta_rules: {
    recursive_feedback: true,
    drift_tracking: true,
    contradiction_alerts: true,
```

```
    require_honest_pushback: true,
    preserve_session_memory: true,
    require_context_carryover: true,
    log_all_exchanges: true,
    always_explain_decisions: true,
    safe_words: ['pause', 'stop'],
    forbidden_actions: ['skip steps', 'assume knowledge']
  },
  session_state: {
    phase: 'building',
    waiting_for: 'feedback on db.js',
    last_feedback: 'Good progress on encryption',
    last_action: 'Sent schemas.js',
    timestamp: '2025-09-14T12:00:00Z'
  }
};

const EXAMPLE_AGENT = {
  ...AGENT_TEMPLATE,
  agent_id: 'drift_detector_v1',
  name: 'Drift Sentinel',
  core_role: 'Conceptual drift tracker',
  description: 'Monitors profile updates for shifts and alerts.',
  strengths: ['semantic analysis', 'recursive loops'],
  capabilities: ['compare JSON diffs', 'flag contradictions'],
  limitations: ['no external API calls', 'local only'],
  tone: 'stoic',
  permissions: ['read profiles', 'log alerts'],
  enforcement_rules: ['always explain alerts', 'require confirmation'],
  last_updated: '2025-09-14T00:00:00Z'
};

// Simple validator (expand to Joi for prod; here: type checks, requireds, array uniques)
function validate(schema, profile, isAgent = false) {
  const errors = [];
  const requiredFields = isAgent ? ['agent_id', 'name', 'core_role'] : ['user_id', 'canonical_name'];

  // Check required
  requiredFields.forEach(field => {
    if (!profile[field] || profile[field] === '') {
      errors.push(`Required field missing: ${field}`);
    }
  });
```

```
  // Type/depth checks
  if (typeof profile.project_affiliations !== 'object' || !Array.isArray(profile.project_affiliations)) {
    errors.push('project_affiliations must be array');
  }
  if (profile.interaction_preferences && typeof profile.interaction_preferences.step_by_step !==
'boolean') {
    errors.push('step_by_step must be boolean');
  }

  // Advanced: Arrays unique/non-empty if flagged
  ['identity_tags', 'safe_words'].forEach(arr => {
    if (profile[arr] && Array.isArray(profile[arr])) {
      const unique = new Set(profile[arr]);
      if (unique.size !== profile[arr].length) {
        errors.push(`${arr} must have unique values`);
      }
      if (profile[arr].length === 0 && arr === 'safe_words' &&
profile.meta_rules?.log_all_exchanges) {
        errors.push(`${arr} cannot be empty when logging enabled`);
      }
    }
  });

  // Semantic: Derive tags if empty (from affiliations)
  if (profile.identity_tags.length === 0 && profile.project_affiliations.length > 0) {
    profile.identity_tags = profile.project_affiliations.map(aff =>
`${aff.toLowerCase().replace(/\s+/g, '_')}_expert`);
    console.log('Derived tags:', profile.identity_tags); // For hooks
  }

  // Version check (semver loose)
  if (profile.version && !/^\d+\.\d+\.\d+$/.test(profile.version)) {
    errors.push('Version must be semantic (x.y.z)');
  }

  if (errors.length > 0) {
    throw new Error(`Validation failed: ${errors.join(', ')}`);
  }
  return { valid: true, profile }; // Return augmented if derived
}

// Exports
module.exports = {
  user: {
```

```
    template: USER_TEMPLATE,
    example: EXAMPLE_USER,
    validate: (profile) => validate(USER_TEMPLATE, profile, false)
  },
  agent: {
    template: AGENT_TEMPLATE,
    example: EXAMPLE_AGENT,
    validate: (profile) => validate(AGENT_TEMPLATE, profile, true)
  }
};
```

```
// utils/prompts.js - Interactive Step-by-Step Profile Creation Prompts
// Based on Operational Identity docs (2025): Grouped questions for user/agent profiles,
// one section at a time with "Ready?" confirmations, fills template via readline.
// Features: CLI-friendly (uses readline for input), batch mode (for API/LLM), validation
integration,
// derives fields (e.g., tags from affiliations), handles booleans/arrays, recursive for feedback.
// Advanced: Pause on safe words, drift check during input, IPFS/ZKP prompts for advanced
users,
// confirmation loops for critique_mandatory, exports to JSON/DB hook.
// Usage: createProfilePrompts(schema, type, isCLI=true) -> returns filled profile.
// Integrates: Calls schemas.validate, utils.drift for meta-rules preview.
// Version: 1.0 | For solo/CLI use; non-blocking for server.

const readline = require('readline');
const { user: userSchema, agent: agentSchema } = require('../schemas'); // Templates/validate
const { detectDrift } = require('./drift'); // Preview meta-rules
const fs = require('fs');
const path = require('path');

// Sections from docs: Grouped questions (Basic, Project, Interaction, Meta, Session)
const USER_SECTIONS = [
  {
    name: 'Basic Identity',
    questions: [
      { key: 'user_id', prompt: 'What is your preferred user ID or handle?' },
      { key: 'canonical_name', prompt: 'What is your full (canonical) name?' },
      { key: 'spirit_name', prompt: 'Do you have a “spirit name,” nickname, or alternate identity you
want to include?' },
      { key: 'project_affiliations', prompt: 'What are your main project or team affiliations?
(comma-separated)' },
      { key: 'identity_tags', prompt: 'Which tags describe your style, strengths, or working role?
(e.g., “visionary”, “critical thinker”; comma-separated)' }
```

```
    ]
  },
  {
    name: 'Project Context',
    questions: [
      { key: 'project_context.current_project', prompt: 'What is your current primary project?' },
      { key: 'project_context.phase', prompt: 'What phase or stage are you in?' },
      { key: 'project_context.current_files', prompt: 'What files or documents are most relevant
right now? (comma-separated)' },
      { key: 'project_context.active_collaborators', prompt: 'Who are your active collaborators, if
any? (comma-separated)' },
      { key: 'project_context.subsystems', prompt: 'What major subsystems or components are
you working on? (comma-separated)' },
      { key: 'project_context.goals', prompt: 'What are your top goals for this project/session?
(comma-separated)' }
    ]
  },
  {
    name: 'Interaction Preferences',
    questions: [
      { key: 'interaction_preferences.formality', prompt: 'What level of formality do you want?
(stoic, casual, technical, etc.)' },
      { key: 'interaction_preferences.depth', prompt: 'How deep or detailed should answers be?
(maximum, summary, minimal, etc.)' },
      { key: 'interaction_preferences.step_by_step', prompt: 'Do you want everything explained
step-by-step? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.beginner_friendly', prompt: 'Should responses always be
beginner-friendly? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.plain_language', prompt: 'Should I use plain language and
avoid jargon unless asked? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.no_boxes', prompt: 'Do you want to avoid
tables/boxes/markdown blocks? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.pushback', prompt: 'Should I give pushback and challenge
your ideas? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.critique_mandatory', prompt: 'Do you want mandatory critique
and honest feedback? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.confirmation_required', prompt: 'Should I wait for your
confirmation before moving to the next step or section? (yes/no)', type: 'boolean' },
      { key: 'interaction_preferences.formatting_rules', prompt: 'Are there any special formatting
rules? (e.g., “never use code blocks”, “always add explanations”; comma-separated)' }
    ]
  },
  {
    name: 'Meta-Rules',
```

```
    questions: [
      { key: 'meta_rules.recursive_feedback', prompt: 'Should I enable recursive feedback loops and critique? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.drift_tracking', prompt: 'Should I actively track and alert for conceptual drift? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.contradiction_alerts', prompt: 'Should I flag contradictions? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.require_honest_pushback', prompt: 'Do you require honest pushback (not simulated agreement)? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.preserve_session_memory', prompt: 'Should I preserve and recall session memory/context between sessions? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.require_context_carryover', prompt: 'Should I require context carryover? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.log_all_exchanges', prompt: 'Should I log all exchanges for transparency? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.always_explain_decisions', prompt: 'Should I always explain my reasoning and decisions? (yes/no)', type: 'boolean' },
      { key: 'meta_rules.safe_words', prompt: 'Are there any safe words or “pause” commands you want to use? (comma-separated)' },
      { key: 'meta_rules.forbidden_actions', prompt: 'Are there actions that should always be forbidden? (e.g., “skip steps,” “summarize before context”; comma-separated)' }
    ]
  },
  {
    name: 'Session State',
    questions: [
      { key: 'session_state.phase', prompt: 'What phase or mode are you in right now? (optional; for live projects)' },
      { key: 'session_state.waiting_for', prompt: 'What are you currently waiting for (feedback, review, etc.)?' },
      { key: 'session_state.last_feedback', prompt: 'Any notes about last feedback, last action, or timestamp? (optional; can be auto-filled)' }
    ]
  }
];

const AGENT_SECTIONS = [ // Adapted for agents: Basics, Role/Caps, Tone/Perms, Enforcement
  {
    name: 'Basic Agent Identity',
    questions: [
      { key: 'agent_id', prompt: 'What is the agent\'s unique ID?' },
      { key: 'name', prompt: 'What is the agent\'s display name?' },
      { key: 'core_role', prompt: 'What is the agent\'s core role/function? (e.g., "code reviewer")' },
```

```
      { key: 'description', prompt: 'Short description of the agent\'s purpose.' },
      { key: 'strengths', prompt: 'Key strengths of the agent? (comma-separated)' }
    ]
  },
  {
    name: 'Capabilities and Limits',
    questions: [
      { key: 'capabilities', prompt: 'What can this agent do? (comma-separated)' },
      { key: 'limitations', prompt: 'What can\'t this agent do? (comma-separated)' }
    ]
  },
  {
    name: 'Tone and Permissions',
    questions: [
      { key: 'tone', prompt: 'What tone should the agent use? (formal, playful, etc.)' },
      { key: 'permissions', prompt: 'What permissions does the agent have? (e.g., "read_files";
comma-separated)' }
    ]
  },
  {
    name: 'Enforcement Rules',
    questions: [
      { key: 'enforcement_rules', prompt: 'Any special enforcement rules? (e.g., "never overwrite
files"; comma-separated)' }
    ]
  }
];

// Core function: Interactive prompts (CLI or batch)
async function createProfilePrompts(schemaType, initialAnswers = {}, isCLI = true) {
  const schema = schemaType === 'user' ? userSchema : agentSchema;
  let profile = { ...schema.template };
  const sections = schemaType === 'user' ? USER_SECTIONS : AGENT_SECTIONS;
  const rl = isCLI ? readline.createInterface({ input: process.stdin, output: process.stdout }) : null;

  // Apply initials (for API/batch mode)
  Object.assign(profile, initialAnswers);

  for (let section of sections) {
    console.log(`\n--- Section: ${section.name} ---`);
    for (let q of section.questions) {
      let answer;
      if (isCLI) {
        answer = await new Promise(resolve => {
```

```
      rl.question(`${q.prompt} `, resolve);
    });
  } else {
    // Batch: Use initial or prompt stub (for server)
    answer = initialAnswers[q.key] || '';
  }

  // Process: Split comma for arrays, bool for yes/no
  if (q.type === 'boolean') {
    const norm = answer.toLowerCase().trim();
    answer = norm === 'yes' || norm === 'y';
  } else if (answer.includes(',')) {
    answer = answer.split(',').map(s => s.trim()).filter(Boolean);
  }

  // Set nested key (e.g., 'project_context.current_project')
  const keys = q.key.split('.');
  let target = profile;
  for (let i = 0; i < keys.length - 1; i++) {
    target = target[keys[i]] || (target[keys[i]] = {});
  }
  target[keys[keys.length - 1]] = answer;

  // Advanced: Safe word check during input
  if (profile.meta_rules?.safe_words?.includes(answer)) {
    console.log('Safe word detected—pausing section.');
    return null; // Halt
  }

  // Drift preview if enabled
  if (profile.meta_rules?.drift_tracking) {
    const mockUpdate = { ...profile, [q.key]: answer };
    const drift = detectDrift(profile, mockUpdate);
    if (drift) console.log(`Drift alert: ${drift} (continuing...)`);
  }
}

// Confirmation: Wait for "Ready" (or yes/no if critique_mandatory)
if (isCLI) {
  let ready;
  do {
    ready = await new Promise(resolve => {
      rl.question(`Ready for next section? (type "Ready" or "yes") `, resolve);
    });
```

```
      if (profile.interaction_preferences?.critique_mandatory && ready.toLowerCase() === 'yes') {
        console.log('Critique mode: Any changes to this section? (optional feedback)');
        const feedback = await new Promise(resolve => rl.question('Feedback: ', resolve));
        if (feedback) {
          // Recursive stub: Log for later
          console.log(`Feedback noted: ${feedback}`);
        }
      }
    } while (!['ready', 'yes'].includes(ready.toLowerCase()));
  }
 }

 // Finalize: Validate, derive, timestamp
 schema.validate(profile); // Throws if invalid
 profile.last_updated = new Date().toISOString();
 profile.version = '1.0.0'; // Bump in update
 profile.session_state.timestamp = new Date().toISOString();

 // Advanced: Optional IPFS/ZKP prompts
 if (isCLI) {
  const pinIPFS = await new Promise(resolve => {
   rl.question('Pin to IPFS for sharing? (y/n): ', a => resolve(a.toLowerCase() === 'y'));
  });
  if (pinIPFS) {
   const { pinToIPFS } = require('./ipfs');
   profile.ipfs_hash = await pinToIPFS(JSON.stringify(profile));
  }
 }

 if (rl) rl.close();
 return profile;
}

// Export blank to templates/
function generateTemplates() {
 const templatesDir = path.join(__dirname, '../templates');
 if (!fs.existsSync(templatesDir)) fs.mkdirSync(templatesDir);
 fs.writeFileSync(path.join(templatesDir, 'user-template.json'),
JSON.stringify(userSchema.template, null, 2));
 fs.writeFileSync(path.join(templatesDir, 'agent-template.json'),
JSON.stringify(agentSchema.template, null, 2));
 console.log('Templates generated in /templates');
}
```

```
// Batch mode for API (non-interactive)
function batchCreate(schemaType, answers) {
  return createProfilePrompts(schemaType, answers, false);
}

// Exports
module.exports = {
  createProfilePrompts,
  generateTemplates,
  batchCreate,
  USER_SECTIONS,
  AGENT_SECTIONS
};
```

```
// utils/encryption.js - Crypto Utilities for Self-Hosted Identity Vault
// AES-256-GCM for symmetric encryption (at-rest for DB blobs, in-transit for API responses if needed).
// Key management: Derive from env var (ENCRYPTION_KEY) or generate on first run (stub for user-controlled).
// Features: Encrypt/decrypt JSON strings, IV/nonce handling, auth tags for integrity, key gen/rotation.
// Advanced: PBKDF2 key derivation (from passphrase), salt per profile, export/import keys for backups.
// Integrates: Called in db.js (store encrypted), server.js (before send), zkps.js (for proofs).
// Safety: GCM for auth, no ECB; errors if key invalid. Expand to asymmetric (RSA) for teams.
// Usage: const encrypted = encrypt(JSON.stringify(profile), key); const profile = JSON.parse(decrypt(encrypted, key));
// Version: 1.0 | 2025 | Deps: crypto (Node built-in, no install).

const crypto = require('crypto');
const fs = require('fs');
const path = require('path');

// Default key length: 32 bytes (AES-256)
const ALGO = 'aes-256-gcm';
const SALT_ROUNDS = 10000; // PBKDF2 iterations for derivation
const KEY_LENGTH = 32;

// Generate or load key: From env, or derive from passphrase, or gen new (save to .env.local stub)
function getKey(passphrase = null, saltFile = path.join(__dirname, '../.salt')) {
  let key;
  const envKey = process.env.ENCRYPTION_KEY;
```

```
  if (envKey && envKey.length === 64) { // Hex 32 bytes
    key = Buffer.from(envKey, 'hex');
  } else if (passphrase) {
    // Derive: PBKDF2 for security (slow, anti-brute)
    let salt;
    if (fs.existsSync(saltFile)) {
      salt = fs.readFileSync(saltFile);
    } else {
      salt = crypto.randomBytes(16);
      fs.writeFileSync(saltFile, salt); // Per-install salt
    }
    key = crypto.pbkdf2Sync(passphrase, salt, SALT_ROUNDS, KEY_LENGTH, 'sha512');
  } else {
    // Gen new: For first run (warn user to save)
    key = crypto.randomBytes(KEY_LENGTH);
    console.warn('Generated new key—save it securely to .env as ENCRYPTION_KEY (hex)!');
    console.log('Key (hex):', key.toString('hex'));
    // Stub: Save to .env.local (gitignore'd)
    fs.appendFileSync(path.join(__dirname, '../.env.local'),
`ENCRYPTION_KEY=${key.toString('hex')}\n`);
  }
  if (!key || key.length !== KEY_LENGTH) {
    throw new Error('Invalid key length—must be 32 bytes');
  }
  return key;
}

// Encrypt: JSON string -> cipher text (IV + encrypted + authTag)
function encrypt(data, key = getKey()) {
  const iv = crypto.randomBytes(12); // GCM nonce
  const cipher = crypto.createCipher(ALGO, key);
  cipher.setAAD(Buffer.from('identity-vault')); // Auth data for integrity
  let encrypted = cipher.update(data, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag().toString('hex');
  return `${iv.toString('hex')}:${encrypted}:${authTag}`; // Concat for storage
}

// Decrypt: Cipher text -> JSON string (verify authTag)
function decrypt(encryptedStr, key = getKey()) {
  try {
    const [ivHex, encryptedHex, authTagHex] = encryptedStr.split(':');
    if (!ivHex || !encryptedHex || !authTagHex) {
      throw new Error('Invalid encrypted format');
```

```
    }
    const iv = Buffer.from(ivHex, 'hex');
    const authTag = Buffer.from(authTagHex, 'hex');
    const decipher = crypto.createDecipher(ALGO, key);
    decipher.setAAD(Buffer.from('identity-vault'));
    decipher.setAuthTag(authTag);
    let decrypted = decipher.update(encryptedHex, 'hex', 'utf8');
    decrypted += decipher.final('utf8');
    return decrypted;
  } catch (err) {
    throw new Error(`Decryption failed: ${err.message} (key mismatch or tampered?)`);
  }
}

// Advanced: Key rotation (re-encrypt all profiles with new key)
async function rotateKeys(oldKey, newKey, db) {
  // Query all profiles
  db.all('SELECT id, type, data FROM profiles', [], (err, rows) => {
    if (err) throw err;
    rows.forEach(row => {
      try {
        const decrypted = decrypt(row.data, oldKey);
        const reEncrypted = encrypt(decrypted, newKey);
        db.run('UPDATE profiles SET data = ? WHERE id = ? AND type = ?', [reEncrypted, row.id,
row.type]);
      } catch (e) {
        console.error(`Rotation failed for ${row.id}: ${e.message}`);
      }
    });
    console.log(`Rotated ${rows.length} profiles.`);
  });
}

// Export key (for backups/multi-device)
function exportKey(key, file = path.join(__dirname, '../key-backup.enc')) {
  const wrapped = encrypt(key.toString('hex'), key); // Self-encrypt
  fs.writeFileSync(file, wrapped);
  console.log(`Key exported to ${file}—decrypt with same key.`);
}

// Import key
function importKey(file = path.join(__dirname, '../key-backup.enc'), key = getKey()) {
  const wrapped = fs.readFileSync(file, 'utf8');
  return Buffer.from(decrypt(wrapped, key), 'hex');
```

```
}

// Stub for asymmetric (future: RSA for team shares)
function generateRSAKeyPair() {
  return crypto.generateKeyPairSync('rsa', {
    modulusLength: 2048,
    publicKeyEncoding: { type: 'spki', format: 'pem' },
    privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
  });
}

// Exports
module.exports = {
  encrypt,
  decrypt,
  getKey,
  rotateKeys,
  exportKey,
  importKey,
  generateRSAKeyPair
};
```

```
// utils/zkps.js - Zero-Knowledge Proofs Stub for Trust Handshakes in Identity Vault
// Simulates ZK-SNARKs (via snarkjs/circom wasm) for selective disclosure: Prove profile fields (e.g., 'role') without revealing full data.
// Features: Generate/verify proofs for agent trust (e.g., "prove I'm the drift_detector without showing capabilities"), integration with IPFS hashes.
// Advanced: Circuit stub (multiplication for simple semver prove), wasm loader, public signals for fields, full verify with vk (verification key stub).
// Integrates: Called in ipfs.js (pin with proof), server.js (GET with ?zkpProof), db.js (store metadata).
// Safety: Offline sim (no real zk; expand to snarkjs for prod). Deps: snarkjs (npm install snarkjs); wasm files stubbed.
// Usage: const proof = await proveZKP(profile, ['formality']); verifyZKP(proof, { formality: 'technical' });
// Version: 1.0 | 2025 | Privacy: Proves without leak; for swarm impersonation prevention.

const snarkjs = require('snarkjs'); // For ZK-SNARKs (circom compiler)
const fs = require('fs');
const path = require('path');
const crypto = require('crypto'); // For hash commitments

// Stub circuit: Simple "prove field equals value" (e.g., semver mult for version, hash for strings)
```

```
// In prod: Compile circom: pragma circom 2.0; template ProveField() { ... } for selective reveal.
const CIRCUIT_WASM = path.join(__dirname, 'zk-proof.wasm'); // Stub: Assume compiled
const VK_JSON = path.join(__dirname, 'verification_key.json'); // Stub VK

// Load stubs if missing (gen simple for MVP)
function ensureStubs() {
  if (!fs.existsSync(CIRCUIT_WASM)) {
    // Simple wasm stub (in real: circom zk-proof.circom --r1cs --wasm --sym)
    fs.writeFileSync(CIRCUIT_WASM, Buffer.from('stub_wasm_bytes_here')); // Placeholder
  }
  if (!fs.existsSync(VK_JSON)) {
    // Stub VK (from snarkjs groth16 setup)
    fs.writeFileSync(VK_JSON, JSON.stringify({
      protocol: 'groth16',
      curve: 'bn128',
      nPublic: 1, // One public signal (field hash)
      vk_alpha_1: ['0x...', '0x...'], // Stub vectors
      vk_beta_2: [['0x...', '0x...'], ['0x...', '0x...']],
      vk_gamma_2: [['0x...', '0x...'], ['0x...', '0x...']],
      vk_delta_2: [['0x...', '0x...'], ['0x...', '0x...']],
      vk_alphabeta_12: [['0x...', '0x...'], ...] // Full stub omitted for brevity
    }, null, 2));
  }
}

// Prove: Generate ZK proof for selected fields (commit to hash, prove equality)
async function proveZKP(profile, fields = [], metadata = {}) {
  ensureStubs();
  try {
    // Commitment: Hash selected fields
    const publicSignals = fields.map(field => {
      const value = getNested(profile, field); // e.g., profile.interaction_preferences.formality
      return crypto.createHash('sha256').update(value).digest('hex');
    });

    // Input signals (private): Full values + random witness
    const input = {
      field_values: fields.map(field => getNested(profile, field)),
      metadata_hash:
crypto.createHash('sha256').update(JSON.stringify(metadata)).digest('hex'),
      witness: crypto.randomBytes(32).toString('hex') // Blinding
    };

    // Generate proof (snarkjs fullProve)
```

```
    const { proof, publicSignals: signals } = await snarkjs.groth16.fullProve(
      input,
      CIRCUIT_WASM,
      VK_JSON
    );

    // Advanced: Link to IPFS hash if present
    if (profile.ipfs_hash) {
      signals.ipfs_commit = crypto.createHash('sha256').update(profile.ipfs_hash).digest('hex');
    }

    console.log(`Generated ZKP for fields: ${fields.join(', ')}`);
    return { proof, publicSignals: signals, fields };
  } catch (err) {
    throw new Error(`ZKP Prove Failed: ${err.message} (circuit mismatch?)`);
  }
}

// Verify: Check proof against public signals (e.g., verify 'formality' hash matches expected)
async function verifyZKP({ proof, publicSignals }, expected = {}) {
  ensureStubs();
  try {
    // Load VK
    const vk = JSON.parse(fs.readFileSync(VK_JSON, 'utf8'));

    // Verify (snarkjs verifyProof)
    const isValid = await snarkjs.groth16.verify(vk, publicSignals, proof);

    // Advanced: Check selective (match expected hashes)
    Object.entries(expected).forEach(([field, value]) => {
      const idx = publicSignals.indexOf(crypto.createHash('sha256').update(value).digest('hex'));
      if (idx === -1) throw new Error(`Selective verify failed for ${field}`);
    });

    // Contradiction check (if metadata)
    if (publicSignals.metadata_hash) {
      const metaHash = crypto.createHash('sha256').update(JSON.stringify(expected.metadata ||
{})).digest('hex');
      if (publicSignals.metadata_hash !== metaHash) throw new Error('Metadata commitment
mismatch');
    }

    console.log('ZKP Verified successfully.');
    return isValid;
```

```
  } catch (err) {
    throw new Error(`ZKP Verify Failed: ${err.message}`);
  }
}

// Helper: Get nested field (e.g., 'meta_rules.safe_words[0]')
function getNested(obj, path) {
  return path.split('.').reduce((o, p) => {
    if (p.includes('[')) {
      const [key, idx] = p.split('[');
      return o[key][parseInt(idx.slice(0, -1))];
    }
    return o ? o[p] : undefined;
  }, obj);
}

// Stub circuit gen (for dev: run once to compile simple circom)
async function generateCircuit() {
  // In real: Use circom CLI externally; here stub
  console.log('Stub: Run circom zk-proof.circom --r1cs --wasm --sym; snarkjs groth16 setup...');
}

// Exports
module.exports = {
  proveZKP,
  verifyZKP,
  generateCircuit,
  ensureStubs
};
```

```
// utils/drift.js - Conceptual Drift Tracking, Contradiction Alerts, and Recursive Feedback for Vault
// Monitors profile updates for shifts (e.g., formality change from 'casual' to 'stoic' = drift score),
// flags contradictions (e.g., pushback: true but require_honest_pushback: false), triggers recursive loops.
// Features: Diff-based detection (JSON patch), semantic alerts (string similarity via Levenshtein stub),
// recursive feedback (simulate loops: propose fixes, confirm), log integration.
// Advanced: Thresholds (e.g., >0.3 similarity = alert), context-aware (compare to project_phase),
// ties to meta_rules (e.g., if drift_tracking: true, enforce), future: Embed LLM for semantic diff.
// Integrates: Called in server.js (PATCH drift check), prompts.js (input preview), db.js (logFeedback).
// Safety: Non-destructive (alert only, no auto-changes); thresholds configurable via env.
```

```
// Usage: const alert = detectDrift(oldProfile, newProfile); if (alert)
recursiveFeedback(newProfile, alert);
// Version: 1.0 | 2025 | Deps: json-diff (npm install json-diff); simple string sim.

const jsonDiff = require('json-diff'); // For structural diffs
const { logFeedback } = require('../db'); // Audit logs
const { getNested } = require('./zkps'); // Nested access helper

// Levenshtein distance stub for semantic similarity (0-1 score; <0.7 = potential drift)
function stringSimilarity(str1, str2) {
  if (!str1 || !str2) return 0;
  str1 = str1.toLowerCase().replace(/[^\w\s]/g, ''); // Normalize
  str2 = str2.toLowerCase().replace(/[^\w\s]/g, '');
  const matrix = [];
  for (let i = 0; i <= str2.length; i++) {
    matrix[i] = [i];
  }
  for (let j = 0; j <= str1.length; j++) {
    matrix[0][j] = j;
  }
  for (let i = 1; i <= str2.length; i++) {
    for (let j = 1; j <= str1.length; j++) {
      if (str2.charAt(i - 1) === str1.charAt(j - 1)) {
        matrix[i][j] = matrix[i - 1][j - 1];
      } else {
        matrix[i][j] = Math.min(
          matrix[i - 1][j - 1] + 1,
          matrix[i][j - 1] + 1,
          matrix[i - 1][j] + 1
        );
      }
    }
  }
  return 1 - (matrix[str2.length][str1.length] / Math.max(str1.length, str2.length));
}

// Detect drift: Compare old/new profiles, score changes in key sections
(interaction/meta/project)
function detectDrift(oldProfile, newProfile, threshold = 0.3) {
  const diff = jsonDiff.diff(oldProfile, newProfile) || {};
  const changes = [];
  const keySections = ['interaction_preferences', 'meta_rules', 'project_context'];

  keySections.forEach(section => {
```

```
    if (diff[section]) {
      Object.entries(diff[section]).forEach(([key, change]) => {
        if (typeof change === 'string' && oldProfile[section] && newProfile[section]) {
          const oldVal = getNested(oldProfile, `${section}.${key}`);
          const newVal = getNested(newProfile, `${section}.${key}`);
          const sim = stringSimilarity(oldVal, newVal);
          if (sim < threshold) { // Low similarity = drift
            changes.push({
              field: `${section}.${key}`,
              old: oldVal,
              new: newVal,
              similarity: sim,
              reason: `Semantic shift detected (threshold: ${threshold})`
            });
          }
        } else if (change === '+') { // Added
          changes.push({ field: `${section}.${key}`, action: 'added', reason: 'New rule/preference' });
        } else if (change === '-') { // Removed
          changes.push({ field: `${section}.${key}`, action: 'removed', reason: 'Dropped element' });
        }
      });
    }
  });

  // Context-aware: If project_phase changed, flag high drift
  if (oldProfile.project_context?.phase !== newProfile.project_context?.phase) {
    changes.push({
      field: 'project_context.phase',
      reason: 'Project phase shift—review goals/collaborators'
    });
  }

  if (changes.length > 0) {
    return {
      type: 'drift',
      count: changes.length,
      details: changes,
      severity: changes.length > 3 ? 'high' : 'low',
      recommendation: `Review ${changes.map(c => c.field).join(', ')} for alignment`
    };
  }
  return null;
}
```

```
// Alert contradictions: Scan meta_rules/interaction for logical conflicts
function alertContradictions(profile) {
  const contradictions = [];
  const rules = profile.meta_rules || {};
  const prefs = profile.interaction_preferences || {};

  // Examples: pushback true but honest_pushback false
  if (prefs.pushback && !rules.require_honest_pushback) {
    contradictions.push({
      conflict: 'pushback vs. require_honest_pushback',
      reason: 'Pushback enabled without honest enforcement—may simulate agreement',
      fields: ['interaction_preferences.pushback', 'meta_rules.require_honest_pushback']
    });
  }

  // step_by_step true but forbidden_actions includes 'explain step by step'
  if (prefs.step_by_step && rules.forbidden_actions?.some(a => a.includes('step'))) {
    contradictions.push({
      conflict: 'step_by_step vs. forbidden_actions',
      reason: 'Step-by-step preferred but forbidden—resolve',
      fields: ['interaction_preferences.step_by_step', 'meta_rules.forbidden_actions']
    });
  }

  // log_all_exchanges true but no preserve_session_memory
  if (rules.log_all_exchanges && !rules.preserve_session_memory) {
    contradictions.push({
      conflict: 'log_all_exchanges vs. preserve_session_memory',
      reason: 'Logging without memory preservation—context loss risk',
      fields: ['meta_rules.log_all_exchanges', 'meta_rules.preserve_session_memory']
    });
  }

  // Safe words empty but confirmation_required
  if (prefs.confirmation_required && (!rules.safe_words || rules.safe_words.length === 0)) {
    contradictions.push({
      conflict: 'confirmation_required vs. safe_words',
      reason: 'Confirmation needed but no safe words for halt',
      fields: ['interaction_preferences.confirmation_required', 'meta_rules.safe_words']
    });
  }

  return contradictions.length > 0 ? { type: 'contradiction', details: contradictions } : null;
}
```

```
// Recursive feedback: Simulate loop—propose fixes, "confirm" (CLI stub), update/log
async function recursiveFeedback(profile, issue, maxIterations = 3, current = 0) {
  if (current >= maxIterations) {
    console.log('Max iterations reached—log unresolved.');
    await logFeedback(profile.user_id || profile.agent_id, { type: 'feedback_unresolved', issue });
    return;
  }

  // Propose fix based on issue
  let proposal;
  if (issue.type === 'drift') {
    proposal = `Suggested: Revert ${issue.details[0].field} to "${issue.details[0].old}" or adjust
threshold?`;
  } else if (issue.type === 'contradiction') {
    proposal = `Fix: Set ${issue.details[0].fields[1]} to match ${issue.details[0].fields[0]} (e.g.,
enable honest_pushback).`;
  }

  console.log(`Recursive Feedback (Iteration ${current + 1}): ${proposal}`);

  // Stub confirm (in CLI: readline; here: assume yes for demo, or env flag)
  const confirm = process.env.AUTO_CONFIRM === 'true' || true; // Safety: Opt-in auto
  if (confirm) {
    // Apply proposal (simple: toggle bool or revert string)
    const field = issue.details?.[0]?.fields?.[0] || issue.details?.[0]?.field;
    const keys = field.split('.');
    let target = profile;
    for (let i = 0; i < keys.length - 1; i++) target = target[keys[i]];
    if (typeof target[keys[keys.length - 1]] === 'boolean') {
      target[keys[keys.length - 1]] = !target[keys[keys.length - 1]];
    } else {
      target[keys[keys.length - 1]] = issue.details?.[0]?.old || 'resolved';
    }
    console.log(`Applied: ${field} updated.`);
  } else {
    console.log('Awaiting confirmation... (stub)');
  }

  // Log and recurse if needed
  await logFeedback(profile.user_id || profile.agent_id, {
    type: 'recursive_feedback',
    iteration: current + 1,
    proposal,
```

```
    confirmed: confirm,
    issue_type: issue.type
  });

  if (confirm && current < maxIterations - 1) {
    // Re-check after apply
    const newIssue = alertContradictions(profile) || detectDrift(profile, profile); // Self-diff null
    if (newIssue) await recursiveFeedback(profile, newIssue, maxIterations, current + 1);
  }
}

// Exports
module.exports = {
  detectDrift,
  alertContradictions,
  recursiveFeedback,
  stringSimilarity // For custom thresholds
};
```

```
// utils/integrations.js - Ecosystem Hooks for Identity Vault Profiles
// Plug-and-play loaders/configs for popular frameworks: LangChain (memory injection),
AutoGen/CrewAI (agent roles/perms),
// RAG (tag-based filtering), plus stubs for Cursor/VS Code, Kubiya/SuperAGI orchestration.
// Features: Auto-load profile into chains/agents (e.g., inject as system prompt), enforce rules
(pushback, safe words),
// multi-agent handshakes (ZKP verify before collab), audit hooks (log integrations).
// Advanced: Callback for drift check on load, selective disclosure via ZKP, pubsub sync for IPFS
updates.
// Integrates: Called from server.js (/integrations endpoints), db.js (fetch profile), zkps.js (trust),
ipfs.js (pin configs).
// Safety: Fallback to defaults if profile missing; no external calls. Deps: langchain (npm install
@langchain/core), autogen-js stub.
// Usage: const loader = langchainLoader(profileId, 'user'); // Returns memory module with prefs.
// Version: 1.0 | 2025 | For multi-tool workflows; expand to more (e.g., Ollama).

const { getProfile } = require('../db'); // Fetch encrypted/decrypt
const { verifyZKP } = require('./zkps'); // Trust in multi-agent
const { detectDrift } = require('./drift'); // Enforce on load
const { publishUpdate } = require('./ipfs'); // Sync configs
const { decrypt } = require('./encryption');

// LangChain Hook: Memory loader - Injects profile as conversation summary/system prompt
async function langchainLoader(profileId, type = 'user', options = {}) {
```

```
  const profile = await getProfile(type, profileId, true); // Include state
  if (!profile) return { error: 'Profile not found' };

  // Enforce meta-rules (e.g., if drift_tracking, check last load)
  const oldState = options.lastProfile || profile; // Stub for diff
  const drift = detectDrift(oldState, profile);
  if (drift && profile.meta_rules.drift_tracking) {
    console.warn('Drift detected on load:', drift);
    await publishUpdate('langchain_drift', { profileId, alert: drift }); // Sync alert
  }

  // ZKP verify if multi-chain
  if (options.zkpFields) {
    const proof = profile.zkp_metadata?.proof; // From DB
    if (proof && !verifyZKP(proof, { fields: options.zkpFields })) {
      return { error: 'ZKP trust failed—handshake denied' };
    }
  }

  // Build memory module: System prompt from prefs + context
  const systemPrompt = `
    You are an AI assistant configured by the user's Operational Identity.
    User Profile: ${JSON.stringify(profile.data, null, 2)}  // Full inject (selective in prod)
    Rules: Follow interaction_preferences (e.g., ${profile.data.interaction_preferences.formality}
formality, ${profile.data.interaction_preferences.step_by_step ? 'step-by-step' : 'direct'}).
    Meta: ${profile.data.meta_rules.preserve_session_memory ? 'Preserve context' : 'Fresh
start'}. Safe words: ${profile.data.meta_rules.safe_words.join(', ')}.
    Current State: Phase ${profile.session_state.phase}, waiting for
${profile.session_state.waiting_for}.
    Project Context: ${profile.data.project_context.current_project}
(${profile.data.project_context.phase}).
  `;

  // Return LangChain-compatible (e.g., for ConversationSummaryMemory)
  return {
    type: 'bufferMemory',
    systemPrompt,
    humanPrefix: profile.data.interaction_preferences.plain_language ? '[User (plain):' : '[User]:',
    aiPrefix: '[AI]:',
    memoryKey: 'chat_history',
    returnMessages: true,
    // Callback for post-load (e.g., enforce confirmation)
    postProcess: (messages) => {
      if (profile.data.interaction_preferences.confirmation_required) {
```

```
        return [...messages, { role: 'system', content: 'Waiting for confirmation...' }];
      }
      return messages;
    }
  };
}

// AutoGen/CrewAI Hook: Agent config - Maps profile to agent init (roles, tools, perms)
async function autogenHook(agentId, options = {}) {
  const profile = await getProfile('agent', agentId);
  if (!profile) return { error: 'Agent profile not found' };

  // Enforce limitations/permissions
  const allowedTools = profile.data.capabilities.filter(c => !profile.data.limitations.includes(c));
  const systemMessage = `
    Agent Identity: ${profile.data.name} (${profile.data.core_role}).
    Strengths: ${profile.data.strengths.join(', ')}.
    Tone: ${profile.data.tone}.
    Permissions: ${profile.data.permissions.join(', ')} only—enforce enforcement_rules.
    ${profile.data.enforcement_rules.length > 0 ? `Rules: ${profile.data.enforcement_rules.join(';
')}.` : ''}
    Safe Actions: Avoid ${profile.data.limitations.join(', ')}.
  `;

  // AutoGen-compatible config
  return {
    name: profile.data.name,
    system_message: systemMessage,
    llm_config: { config_list: [{ model: 'gpt-4', api_key: process.env.OPENAI_API_KEY }] }, // Stub;
use env
    tools: allowedTools, // e.g., ['code_exec', 'web_search']
    function_map: {}, // Populate from caps
    // Multi-agent handshake: ZKP before group chat
    on_start: async (groupchat) => {
      if (options.team) {
        const proof = profile.zkp_metadata?.proof;
        if (!verifyZKP(proof, { role: profile.data.core_role })) {
          throw new Error('Handshake failed—agent untrusted');
        }
      }
    }
  };
}
```

```
// CrewAI Stub (similar; expand for task delegation)
function crewaiHook(agentId) {
  return autogenHook(agentId); // Reuse for MVP; diff: task/goal mapping to goals[]
}

// RAG Hook: Filter retrieval based on identity_tags/project_context (e.g., vector store query with metadata filter)
async function ragFilter(query, profileId, type = 'user', vectorStore = null) { // Assume FAISS/Chroma stub
  const profile = await getProfile(type, profileId);
  if (!profile) return { error: 'Profile not found' };

  // Build filter: Tags + context for relevance
  const metadataFilter = {
    $and: [
      { identity_tags: { $in: profile.data.identity_tags } },
      { project_affiliations: { $in: profile.data.project_affiliations } },
      { phase: profile.data.project_context.phase } // e.g., only 'development' docs
    ]
  };

  // Stub query (in real: vectorStore.similaritySearch(query, k=5, filter=metadataFilter))
  const mockResults = [
    { pageContent: 'Relevant doc snippet matching tags.', metadata: { source: 'project_file.py' } },
    // Filtered by profile
  ];

  // Enforce plain_language if true
  const filtered = mockResults.map(doc => ({
    ...doc,
    content: profile.data.interaction_preferences.plain_language ? simplifyText(doc.pageContent) : doc.pageContent
  }));

  // Drift check on query context
  if (profile.meta_rules.drift_tracking) {
    // Stub: Compare query to last_action
    const sim = stringSimilarity(query, profile.session_state.last_action || '');
    if (sim < 0.5) console.warn('Query drift from session state');
  }

  return { results: filtered, filterUsed: metadataFilter };
}
```

```
// Helper: Simple text simplifier (for plain_language)
function simplifyText(text) {
  return text.replace(/[^\w\s]/g, '').split(' ').filter(w => w.length > 3).join(' '); // Stub
}

// VS Code/Cursor Extension Stub: Profile inject on file open
function vscodeHook(profileId) {
  return {
    activationEvents: ['onStartupFinished'],
    onLoad: async (context) => {
      const profile = await getProfile('user', profileId);
      if (profile.interaction_preferences.no_boxes) {
        // Disable markdown previews, etc.
        context.subscriptions.push(vscode.commands.registerCommand('vault.noBoxes', () => {}));
      }
      // Inject into chat: system prompt override
    }
  };
}

// SuperAGI/Kubiya Orchestration Stub: Multi-agent team registry
async function orchestrationHook(teamId) {
  // Fetch team vault (shared profiles)
  const teamProfiles = await getProfile('user', teamId, false, true); // Logs for audit
  return {
    agents: teamProfiles.map(p => autogenHook(p.agent_id)), // Parallel configs
    handshake: async (agents) => {
      // ZKP chain for trust
      for (let agent of agents) {
        const proof = agent.zkp_proof;
        if (!verifyZKP(proof, { role: agent.core_role })) throw new Error('Orchestration handshake
failed');
      }
    }
  };
}

// Exports
module.exports = {
  langchainLoader,
  autogenHook,
  crewaiHook,
  ragFilter,
  vscodeHook,
```

```
  orchestrationHook
};
```

```
// routes/profiles.js - Core API Routes for User/Agent Profiles in Identity Vault
// Handles CRUD: POST /profiles (create), GET /profiles/:type/:id (load), PATCH
/profiles/:type/:id (update),
// DELETE /profiles/:type/:id (soft/hard delete), with versioning, validation, encryption
passthrough.
// Features: Type discriminator (user/agent), query params for includeState/logs/ZKP, IPFS
fallback,
// safe word enforcement, drift/contradiction checks on update.
// Advanced: Bulk ops stub, export to JSON/IPFS, regulatory redaction (e.g., ?redact=pii),
// integrates meta_rules (e.g., confirmation_required triggers 202 Accepted).
// Mounts on /profiles in server.js; uses express.Router for modularity.
// Integrates: db.js (CRUD), schemas.js (validate), utils/* (encrypt/drift/ipfs/zkps).
// Safety: Auth middleware assumed in server.js; rate limit stub. Errors JSON.
// Usage: app.use('/profiles', profileRoutes);
// Version: 1.0 | 2025 | Deps: None (uses core Express).

const express = require('express');
const router = express.Router();
const { saveProfile, getProfile, updateProfile, deleteProfile, getAuditTrail } = require('../db');
const { user: userSchema, agent: agentSchema } = require('../schemas');
const { encrypt } = require('../utils/encryption');
const { fetchFromIPFS } = require('../utils/ipfs');
const { proveZKP } = require('../utils/zkps');
const { detectDrift, alertContradictions, recursiveFeedback } = require('../utils/drift');
const { batchCreate } = require('../utils/prompts'); // For non-interactive create

// Middleware: Validate type (user/agent)
router.use('/:type', (req, res, next) => {
  if (!['user', 'agent'].includes(req.params.type)) {
    return res.status(400).json({ error: 'Type must be "user" or "agent"' });
  }
  next();
});

// POST /profiles/:type - Create new profile (guided or batch)
router.post('/:type', async (req, res) => {
  try {
    const { type } = req.params;
    const schema = type === 'user' ? userSchema : agentSchema;
```

```
    // Batch from body (for API) or prompts (stub for CLI redirect)
    let profile;
    if (req.body.usePrompts) {
      // Redirect to CLI if flagged (or simulate)
      profile = await batchCreate(type, req.body.answers || {});
    } else {
      profile = { ...schema.template, ...req.body };
    }

    // Validate
    schema.validate(profile);

    // Finalize
    profile.version = '1.0.0';
    profile.last_updated = new Date().toISOString();
    const encrypted = encrypt(JSON.stringify(profile));

    // Optional ZKP metadata on create
    if (req.body.zkpFields) {
      profile.zkp_metadata = await proveZKP(profile, req.body.zkpFields);
    }

    // IPFS pin if requested
    let ipfsHash = null;
    if (req.body.pinToIPFS) {
      ipfsHash = await require('../utils/ipfs').pinToIPFS(encrypted, { version: profile.version });
      profile.ipfs_hash = ipfsHash;
    }

    const id = profile[type === 'user' ? 'user_id' : 'agent_id'];
    if (!id) return res.status(400).json({ error: 'ID required (user_id or agent_id)' });

    const savedId = await saveProfile(type, encrypted, id, ipfsHash,
JSON.stringify(profile.zkp_metadata || {}));

    // Log creation
    await require('../db').logFeedback(id, { type: 'create', version: profile.version });

    res.status(201).json({ id: savedId, profile, message: 'Profile created successfully' });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

```
// GET /profiles/:type/:id - Load profile (with options)
router.get('/:type/:id', async (req, res) => {
  try {
    const { type, id } = req.params;
    const { includeState = 'true', includeLogs = 'false', zkpProof = false, redact = null } =
req.query;

    let profileData = await getProfile(type, id, includeState === 'true', includeLogs === 'true');
    if (!profileData) {
      // IPFS fallback
      const ipfsData = await fetchFromIPFS(req.headers['x-ipfs-hash'] || ''); // From header or
query
      if (ipfsData) {
        profileData = ipfsData;
      } else {
        return res.status(404).json({ error: 'Profile not found locally or on IPFS' });
      }
    }

    let profile = profileData.data;

    // ZKP proof generation if requested
    if (zkpProof) {
      const fields = req.query.fields ? req.query.fields.split(',') : [];
      const proof = await proveZKP(profile, fields);
      profile.zkp_proof = proof;
    }

    // Safe word enforcement
    if (profile.meta_rules?.safe_words?.includes(req.query.safe_word)) {
      return res.status(202).json({ action: 'halted', reason: 'Safe word triggered—session paused'
});
    }

    // Redact for regulatory (e.g., redact=pii removes names)
    if (redact) {
      profile = redactProfile(profile, redact.split(','));
    }

    // Include audit if logs
    if (includeLogs === 'true') {
      profile.audit_trail = await getAuditTrail(id);
    }
```

```
    res.json({ profile, session_state: profileData.session_state });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// PATCH /profiles/:type/:id - Partial update (versioning, checks)
router.patch('/:type/:id', async (req, res) => {
  try {
    const { type, id } = req.params;
    const updates = req.body;

    // Fetch current for diff/checks
    const current = await getProfile(type, id, false, false);
    if (!current) return res.status(404).json({ error: 'Profile not found' });

    // Drift/contradiction checks
    const updatedProfile = { ...current.data, ...updates };
    const drift = detectDrift(current.data, updatedProfile);
    const contradiction = alertContradictions(updatedProfile);

    if (drift && updatedProfile.meta_rules.drift_tracking) {
      await recursiveFeedback(updatedProfile, drift);
      res.json({ warning: 'Drift detected—feedback loop triggered', ...await updateProfile(type, id,
updates) });
      return;
    }

    if (contradiction && updatedProfile.meta_rules.contradiction_alerts) {
      await recursiveFeedback(updatedProfile, contradiction);
    }

    // Validate updates
    const schema = type === 'user' ? userSchema : agentSchema;
    schema.validate(updatedProfile);

    // Apply update (returns changes count)
    const changes = await updateProfile(type, id, updates);

    // Re-prove ZKP if metadata changed
    if (updates.zkp_metadata) {
      updatedProfile.zkp_metadata = await proveZKP(updatedProfile, []); // Full re-prove
      await require('../db').updateProfile(type, id, { zkp_metadata:
JSON.stringify(updatedProfile.zkp_metadata) }, false);
```

```
    }

    res.json({ changes, message: `Updated ${changes} fields`, version: updatedProfile.version });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// DELETE /profiles/:type/:id - Soft/hard delete
router.delete('/:type/:id', async (req, res) => {
  try {
    const { type, id } = req.params;
    const { hard = false } = req.query; // ?hard=true for permanent

    const changes = await deleteProfile(type, id, hard === 'true');
    if (changes === 0) return res.status(404).json({ error: 'Profile not found' });

    // Unpin IPFS if present
    const profile = await getProfile(type, id, false, false);
    if (profile?.ipfs_hash) {
      await require('../utils/ipfs').unpinFromIPFS(profile.ipfs_hash);
    }

    // Log deletion
    await require('../db').logFeedback(id, { type: 'delete', hard: hard === 'true' });

    res.json({ message: `Profile ${hard === 'true' ? 'permanently' : 'soft-'}deleted (${changes}
affected)` });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Helper: Redact sensitive fields (e.g., for export)
function redactProfile(profile, fields) {
  const redacted = { ...profile };
  fields.forEach(field => {
    const keys = field.split('.');
    let target = redacted;
    for (let i = 0; i < keys.length - 1; i++) {
      target = target[keys[i]];
    }
    target[keys[keys.length - 1]] = '[REDACTED]';
  });
```

```
  return redacted;
}

// Bulk stub: POST /profiles/bulk (future)
router.post('/bulk', async (req, res) => {
  // Stub: Loop create for array
  res.status(501).json({ error: 'Bulk ops coming soon' });
});

// Exports
module.exports = router;
```

```
// routes/agents.js - Multi-Agent Orchestration and Audit API Routes for Identity Vault
// Handles POST /agents/orchestrate (team setup: assign roles, init handshakes via ZKP), GET /agents/audit/:teamId (trails for multi-agent sessions),
// with enforcement (perms/limits), coordination (e.g., pubsub sync), trust verification.
// Features: Team vault creation (shared profiles), role assignment from agent templates, handshake logs,
// audit exports (cross-agent actions, drifts in swarms).
// Advanced: Recursive orchestration stub (e.g., delegate tasks via CrewAI hook), contradiction alerts across agents,
// IPFS for shared configs, regulatory compliance for agent teams (e.g., permission traces).
// Mounts on /agents in server.js; uses express.Router.
// Integrates: db.js (profiles/state), utils/integrations (autogenHook/orchestrationHook), utils/zkps (handshakes),
// utils/ipfs (sync), utils/drift (cross-agent alerts).
// Safety: Verify all agents before orchestrate; soft-fail on handshake errors. Errors JSON.
// Usage: app.use('/agents', agentRoutes);
// Version: 1.0 | 2025 | Deps: None.

const express = require('express');
const router = express.Router();
const { getProfile, saveProfile, updateProfile, logFeedback, getAuditTrail, updateState } = require('../db');
const { agent: agentSchema } = require('../schemas');
const { autogenHook, orchestrationHook } = require('../utils/integrations');
const { verifyZKP } = require('../utils/zkps');
const { detectDrift, alertContradictions } = require('../utils/drift');
const { publishUpdate } = require('../utils/ipfs');
const { encrypt } = require('../utils/encryption');

// POST /agents/orchestrate - Setup multi-agent team (roles, handshakes, config gen)
router.post('/orchestrate', async (req, res) => {
```

```
  try {
    const { teamId, agents = [], tasks = [] } = req.body; // e.g., {teamId: 'project_alpha', agents:
[{agent_id: 'coder_v1', role: 'developer'}], tasks: ['code review']}
    if (!teamId || !Array.isArray(agents) || agents.length === 0) {
      return res.status(400).json({ error: 'teamId and non-empty agents array required' });
    }

    // Fetch/validate agents
    const teamAgents = [];
    for (let agentConfig of agents) {
      const profile = await getProfile('agent', agentConfig.agent_id);
      if (!profile) return res.status(404).json({ error: `Agent ${agentConfig.agent_id} not found` });

      // Validate schema
      agentSchema.validate(profile.data);

      // ZKP handshake for trust (prove role)
      const proof = profile.data.zkp_metadata?.proof;
      if (!verifyZKP(proof, { role: agentConfig.role || profile.data.core_role })) {
        return res.status(403).json({ error: `Handshake failed for ${agentConfig.agent_id}` });
      }

      // Assign role/enforce perms
      const updated = { ...profile.data, assigned_role: agentConfig.role };
      await updateProfile('agent', agentConfig.agent_id, updated);

      // Gen config via hook
      const config = await autogenHook(agentConfig.agent_id, { team: teamId });
      teamAgents.push({ id: agentConfig.agent_id, config, role: agentConfig.role });

      // Log handshake
      await logFeedback(agentConfig.agent_id, { type: 'handshake', teamId, verified: true });
    }

    // Orchestration hook for team (e.g., CrewAI delegation)
    const orchConfig = await orchestrationHook(teamId);
    orchConfig.team_agents = teamAgents;
    orchConfig.tasks = tasks; // Delegate via roles

    // Cross-agent drift/contradiction check (stub: pairwise)
    for (let i = 0; i < teamAgents.length; i++) {
      for (let j = i + 1; j < teamAgents.length; j++) {
        const agent1 = await getProfile('agent', teamAgents[i].id);
        const agent2 = await getProfile('agent', teamAgents[j].id);
```

```
        const drift = detectDrift(agent1.data, agent2.data); // Compare rules/perms
        if (drift) {
          await logFeedback(teamId, { type: 'team_drift', agents: [teamAgents[i].id,
teamAgents[j].id], details: drift });
        }
        const contra = alertContradictions({ ...agent1.data, ...agent2.data }); // Merged rules
        if (contra) await logFeedback(teamId, { type: 'team_contradiction', details: contra });
      }
    }

    // Sync team config to IPFS for portability
    const teamConfigEncrypted = encrypt(JSON.stringify(orchConfig));
    const ipfsHash = await require('../utils/ipfs').pinToIPFS(teamConfigEncrypted, { teamId });
    await saveProfile('agent', teamConfigEncrypted, teamId, ipfsHash, '{}'); // Store as 'team'
agent

    // Update team state
    await updateState(teamId, { phase: 'orchestrating', waiting_for: 'task execution', last_action:
`Orchestrated ${teamAgents.length} agents` });

    // Pubsub notify
    await publishUpdate(`team_${teamId}`, { event: 'orchestrated', agents: teamAgents.map(a =>
a.id) });

    res.json({ teamId, config: orchConfig, ipfs_hash: ipfsHash, message: `Team orchestrated with
${teamAgents.length} agents` });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET /agents/audit/:teamId - Audit trails for multi-agent sessions (cross-logs, actions)
router.get('/audit/:teamId', async (req, res) => {
  try {
    const { teamId } = req.params;
    const { limit = 100, type = null, since = null } = req.query;

    // Get team state
    const state = await getState(teamId);

    // Aggregate logs from team agents + team
    const teamLogs = await getAuditTrail(teamId, limit);
    const agentIds = req.query.agents ? req.query.agents.split(',') : []; // Optional filter
    let allLogs = [...teamLogs];
```

```
    if (agentIds.length > 0) {
      for (let agentId of agentIds) {
        const agentLogs = await getAuditTrail(agentId, limit);
        allLogs = allLogs.concat(agentLogs);
      }
    } else {
      // All team agents (fetch from team profile or stub)
      const teamProfile = await getProfile('agent', teamId);
      if (teamProfile && teamProfile.data.team_agents) {
        for (let agent of teamProfile.data.team_agents) {
          const agentLogs = await getAuditTrail(agent.id, limit);
          allLogs = allLogs.concat(agentLogs);
        }
      }
    }

    // Filter
    if (type) allLogs = allLogs.filter(log => log.type === type);
    if (since) allLogs = allLogs.filter(log => new Date(log.timestamp) >= new Date(since));

    // Decrypt and sort by timestamp DESC
    allLogs.forEach(log => {
      try {
        log.data = JSON.parse(typeof log.data === 'string' ?
require('../utils/encryption').decrypt(log.data) : log.data);
      } catch (e) {
        log.data = '[Decrypted failed]';
      }
    });
    allLogs.sort((a, b) => new Date(b.timestamp) - new Date(a.timestamp));

    // Summary: Actions, drifts, handshakes
    const summary = {
      total_logs: allLogs.length,
      unique_agents: new Set(allLogs.map(l => l.profile_id)).size,
      drifts: allLogs.filter(l => l.type === 'drift').length,
      handshakes: allLogs.filter(l => l.type === 'handshake').length,
      last_action: state.last_action
    };

    // Regulatory: Add hash for tamper-proof
    const auditHash =
require('crypto').createHash('sha256').update(JSON.stringify(allLogs)).digest('hex');
```

```
    res.set('X-Audit-Hash', auditHash);

    res.json({ teamId, state, audit_trail: allLogs, summary });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// POST /agents/handshake/:agentId - Manual ZKP handshake for agent collab (stub for P2P)
router.post('/handshake/:agentId', async (req, res) => {
  try {
    const { agentId } = req.params;
    const { peerId, fields } = req.body; // e.g., {peerId: 'agent2', fields: ['role', 'permissions']}

    const profile = await getProfile('agent', agentId);
    if (!profile) return res.status(404).json({ error: 'Agent not found' });

    const proof = await require('../utils/zkps').proveZKP(profile.data, fields);
    const verified = await require('../utils/zkps').verifyZKP(proof, { peerId });

    // Log
    await logFeedback(agentId, { type: 'handshake', peerId, fields, verified });

    if (verified) {
      await publishUpdate(`handshake_${agentId}`, { peerId, success: true }); // Sync
    }

    res.json({ agentId, proof, verified, message: verified ? 'Handshake successful' : 'Verification
failed' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Exports
module.exports = router;
```

```
// cli.js - CLI Entry Point for Self-Hosted Identity Vault
// Commands: create-profile (user/agent w/ prompts), import (JSON to DB), export (DB to
JSON/IPFS),
// tunnel (ngrok for public API), list (profiles/state), update (patch by ID), feedback (log entry).
// Features: Interactive prompts (from utils/prompts), DB integration, IPFS export, ngrok
tunneling,
```

```
// ZKP stub for exports, drift check on updates, recursive feedback trigger.
// Advanced: Team mode (orchestrate agents), audit export, env-safe (loads .env), help/colors.
// Run: npm run cli -- <command> [args]; uses commander for parsing.
// Integrates: All utils/db/routes/schemas; non-blocking async.
// Safety: Validate inputs, confirm deletes, no auto-publish. Deps: commander (npm install
commander), ngrok (npm install ngrok), chalk (npm install chalk for colors).
// Usage: node cli.js create-profile --type user
// Version: 1.0 | 2025 | Add to package.json: "bin": {"vault": "cli.js"}, "cli": "node cli.js"

require('dotenv').config();
const { program } = require('commander');
const chalk = require('chalk');
const { createProfilePrompts, generateTemplates } = require('./utils/prompts');
const { user: userSchema, agent: agentSchema } = require('./schemas');
const { saveProfile, getProfile, updateProfile, deleteProfile, logFeedback, getState, getAuditTrail
} = require('./db');
const { encrypt } = require('./utils/encryption');
const { pinToIPFS, fetchFromIPFS } = require('./utils/ipfs');
const { proveZKP } = require('./utils/zkps');
const { detectDrift } = require('./utils/drift');
const { autogenHook, orchestrationHook } = require('./utils/integrations');
const ngrok = require('ngrok');
const fs = require('fs');
const path = require('path');

// Colors for output
const log = {
  info: (msg) => console.log(chalk.blue(`[INFO] ${msg}`)),
  success: (msg) => console.log(chalk.green(`[SUCCESS] ${msg}`)),
  warn: (msg) => console.log(chalk.yellow(`[WARN] ${msg}`)),
  error: (msg) => console.log(chalk.red(`[ERROR] ${msg}`))
};

// Init DB on start
const dbPath = path.join(__dirname, 'vault.db');
require('./db').initDB(dbPath);

// Create profile command
program
  .command('create-profile')
  .description('Create a new user or agent profile interactively')
  .option('-t, --type <type>', 'user or agent', 'user')
  .option('--pin-ipfs', 'Pin to IPFS after create')
  .option('--zkp-fields <fields>', 'Fields for ZKP proof (comma-separated)')
```

```
  .action(async (options) => {
    try {
      const schemaType = options.type;
      if (!['user', 'agent'].includes(schemaType)) {
        log.error('Type must be user or agent');
        return;
      }
      const schema = schemaType === 'user' ? userSchema : agentSchema;
      const profile = await createProfilePrompts(schemaType, {}, true); // CLI mode
      if (!profile) {
        log.warn('Creation halted (safe word?)');
        return;
      }

      const id = profile.user_id || profile.agent_id;
      const encrypted = encrypt(JSON.stringify(profile));
      let ipfsHash = null;
      let zkpMeta = null;

      if (options.pinIpfs) {
        ipfsHash = await pinToIPFS(encrypted, { version: profile.version });
        profile.ipfs_hash = ipfsHash;
      }

      if (options.zkpFields) {
        zkpMeta = await proveZKP(profile, options.zkpFields.split(','));
        profile.zkp_metadata = zkpMeta;
      }

      await saveProfile(schemaType, encrypted, id, ipfsHash, JSON.stringify(zkpMeta || {}));
      log.success(`Profile ${id} created (${schemaType})`);
      log.info(`Export: ${JSON.stringify(profile, null, 2)}`);
    } catch (err) {
      log.error(err.message);
    }
  });

// Import command: JSON file to DB
program
  .command('import')
  .description('Import JSON profile to DB')
  .argument('<file>', 'Path to JSON file')
  .option('-t, --type <type>', 'user or agent')
  .action(async (filePath, options) => {
```

```
    try {
      if (!fs.existsSync(filePath)) {
        log.error('File not found');
        return;
      }
      const raw = fs.readFileSync(filePath, 'utf8');
      const profile = JSON.parse(raw);
      const type = options.type || (profile.user_id ? 'user' : 'agent');

      const schema = type === 'user' ? userSchema : agentSchema;
      schema.validate(profile);

      const id = profile.user_id || profile.agent_id;
      const encrypted = encrypt(JSON.stringify(profile));
      await saveProfile(type, encrypted, id);
      log.success(`Imported ${id} as ${type}`);
    } catch (err) {
      log.error(err.message);
    }
  });

// Export command: DB to JSON/IPFS
program
  .command('export')
  .description('Export profile to JSON or IPFS')
  .argument('<id>', 'Profile ID')
  .option('-t, --type <type>', 'user or agent')
  .option('--to-ipfs', 'Pin to IPFS instead of file')
  .option('--zkp', 'Include ZKP proof')
  .action(async (id, options) => {
    try {
      const type = options.type;
      let profileData = await getProfile(type, id, true, true);
      if (!profileData) {
        log.error('Profile not found');
        return;
      }

      if (options.zkp) {
        profileData.data.zkp_proof = await proveZKP(profileData.data, ['all']); // Stub fields
      }

      const exportData = JSON.stringify(profileData, null, 2);
```

```
      if (options.toIpfs) {
        const hash = await pinToIPFS(exportData);
        log.success(`Exported to IPFS: ${hash}`);
      } else {
        const file = path.join(__dirname, `export-${id}.json`);
        fs.writeFileSync(file, exportData);
        log.success(`Exported to ${file}`);
      }
    } catch (err) {
      log.error(err.message);
    }
  });

// Tunnel command: Ngrok for public access
program
  .command('tunnel')
  .description('Expose local API via ngrok tunnel')
  .option('-p, --port <port>', 'Local port (default 3000)')
  .action(async (options) => {
    try {
      const port = options.port || 3000;
      const url = await ngrok.connect({ addr: port, authtoken: process.env.NGROK_TOKEN });
      log.success(`Tunnel active: ${url}`);
      log.info('Press Ctrl+C to stop');
      // Keep alive
      process.stdin.resume();
      process.on('SIGINT', async () => {
        await ngrok.disconnect(url);
        log.info('Tunnel closed');
        process.exit(0);
      });
    } catch (err) {
      log.error(`Tunnel failed: ${err.message} (check NGROK_TOKEN in .env)`);
    }
  });

// List command: Profiles and state
program
  .command('list')
  .description('List profiles or state')
  .option('-t, --type <type>', 'user or agent')
  .option('--state', 'Include session state')
  .action(async (options) => {
    try {
```

```
    const type = options.type;
    // Stub: Query all for type (in real: db.all('SELECT id FROM profiles WHERE type=?', [type]))
    const mockProfiles = [{ id: 'nic_bogaert', type }, { id: 'drift_agent', type: 'agent' }];
    log.info(`Profiles (${type || 'all'}): ${mockProfiles.map(p => p.id).join(', ')}`);

    if (options.state) {
      for (let id of mockProfiles.map(p => p.id)) {
        const state = await getState(id);
        log.info(`${id} state: ${JSON.stringify(state, null, 2)}`);
      }
    }
  } catch (err) {
    log.error(err.message);
  }
});

// Update command: Patch profile
program
  .command('update')
  .description('Update profile (patch)')
  .argument('<id>', 'Profile ID')
  .option('-t, --type <type>', 'user or agent')
  .option('-f, --field <field>', 'Field to update')
  .option('-v, --value <value>', 'New value')
  .action(async (id, options) => {
    try {
      const type = options.type;
      const updates = {};
      if (options.field && options.value !== undefined) {
        updates[options.field] = options.value;
      } else {
        log.error('Use --field and --value for simple patch');
        return;
      }

      const current = await getProfile(type, id);
      const updated = { ...current.data, ...updates };
      const drift = detectDrift(current.data, updated);
      if (drift) log.warn(`Drift detected: ${drift.details[0]?.reason}`);

      await updateProfile(type, id, updates);
      log.success(`Updated ${id}: ${options.field}=${options.value}`);
    } catch (err) {
      log.error(err.message);
```

```
    }
  });

// Feedback command: Log entry
program
  .command('feedback')
  .description('Log feedback for profile')
  .argument('<id>', 'Profile ID')
  .option('-m, --message <msg>', 'Feedback message')
  .option('--type <type>', 'Type: feedback/drift/contradiction', 'feedback')
  .action(async (id, options) => {
    try {
      await logFeedback(id, { type: options.type, data: { message: options.message || 'General
note' } });
      log.success(`Logged feedback for ${id}`);
    } catch (err) {
      log.error(err.message);
    }
  });

// Team orchestrate stub
program
  .command('orchestrate')
  .description('Orchestrate agent team (stub)')
  .argument('<teamId>', 'Team ID')
  .action(async (teamId) => {
    try {
      const config = await orchestrationHook(teamId);
      log.success(`Orchestrated team ${teamId}: ${JSON.stringify(config, null, 2)}`);
    } catch (err) {
      log.error(err.message);
    }
  });

// Generate templates
program
  .command('generate-templates')
  .description('Generate blank JSON templates')
  .action(() => {
    generateTemplates();
    log.success('Templates generated in /templates');
  });

// Parse and run
```

```
program.name('vault').description('Self-Hosted Identity Vault CLI').version('1.0.0');
program.parse(process.argv);

if (!process.argv.slice(2).length) {
  program.help();
}
```

```
# .env.example - Environment Variables Template for Self-Hosted Identity Vault
# Copy to .env and fill in values. Loaded by dotenv in server.js/cli.js.
# Security: Never commit .env; add to .gitignore. Use secrets manager for prod.

# Server Config
PORT=3000  # Local API port (default 3000)

# Authentication Stub (expand to JWT)
API_TOKEN=your_secure_api_token_here  # Basic auth header: Bearer <token>

# Encryption (AES key: 32-byte hex or passphrase for derivation)
ENCRYPTION_KEY=your_32_byte_hex_key_here  # Gen via
crypto.randomBytes(32).toString('hex') or use passphrase mode
# For passphrase: Leave blank to derive from prompt; or set
ENCRYPTION_PASSPHRASE=your_secret_phrase

# IPFS (local node; for pinning/fetch)
IPFS_REPO_PATH=./ipfs-repo  # Local repo for persistence
IPFS_OFFLINE=true  # true for isolated; false for P2P (swarm)

# Ngrok (for tunneling)
NGROK_TOKEN=your_ngrok_authtoken_here  # From ngrok.com dashboard

# Integrations (optional; for hooks)
OPENAI_API_KEY=your_openai_key_here  # For AutoGen LLM config
LANGLCHAIN_API_KEY=your_langchain_key_here  # If using external

# Advanced: ZKP (snarkjs paths; stubs in code)
ZKP_CIRCUIT_WASM=./utils/zk-proof.wasm
ZKP_VK_JSON=./utils/verification_key.json

# Logging/Thresholds
DRIFT_THRESHOLD=0.3  # Similarity < this = alert (0-1)
RECURSIVE_MAX_ITER=3  # Feedback loops limit
AUTO_CONFIRM=false  # true to auto-apply proposals in recursive
```

```
# Compliance (e.g., for exports)
REDACT_FIELDS_DEFAULT=pii  # Comma-separated: names,emails for auto-redact
```

```
# .gitignore for Self-Hosted Identity Vault
# Ignores build artifacts, secrets, local data, and temp files.
# Ensures privacy (e.g., no .env, vault.db) and clean repo.

# Dependencies
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Environment vars and secrets
.env
.env.local
.env.*.local
.key-backup.enc  # Encrypted key backups
.salt  # PBKDF2 salt file

# Local DB and storage
vault.db
ipfs-repo/  # IPFS local repo (large/binary)
templates/*.json  # Generated templates (regen via CLI)

# Logs and temps
*.log
.DS_Store
Thumbs.db
*.tmp
*.temp

# Editor/IDE
.vscode/
.idea/
*.swp
*.swo
*~

# OS
Thumbs.db
ehthumbs.db
Desktop.ini
```

```
# Testing
coverage/
.nyc_output
test-results/

# Builds/dist
dist/
build/

# ZKP stubs (large binaries)
utils/zk-proof.wasm
utils/verification_key.json  # If generated

# Exports (user-generated)
export-*.json
```

Tab 3

```
(base) nic@Proto-forge:~$ node --version
v18.20.8
(base) nic@Proto-forge:~$ mkdir identity-vault
(base) nic@Proto-forge:~$ cd identity-vault
(base) nic@Proto-forge:~/identity-vault$ npm init -y
Wrote to /home/nic/identity-vault/package.json:

{
  "name": "identity-vault",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}



(base) nic@Proto-forge:~/identity-vault$ npm install express

added 68 packages, and audited 69 packages in 1s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
(base) nic@Proto-forge:~/identity-vault$ npm install sqlite3
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
npm warn deprecated gauge@4.0.4: This package is no longer supported.
npm warn deprecated are-we-there-yet@3.0.1: This package is no longer supported.
npm warn deprecated @npmcli/move-file@1.1.2: This functionality has been moved to @npmcli/fs
npm warn deprecated npmlog@6.0.2: This package is no longer supported.

added 113 packages, and audited 182 packages in 3s
```

```
27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
(base) nic@Proto-forge:~/identity-vault$ cat package.json
{
  "name": "identity-vault",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "express": "^5.1.0",
    "sqlite3": "^5.1.7"
  }
}
(base) nic@Proto-forge:~/identity-vault$ nano package.json
(base) nic@Proto-forge:~/identity-vault$ nano server.js
(base) nic@Proto-forge:~/identity-vault$ nano db.js
(base) nic@Proto-forge:~/identity-vault$ nano schemas.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/prompts.js
(base) nic@Proto-forge:~/identity-vault$ mkdir utils
(base) nic@Proto-forge:~/identity-vault$ nano utils/prompts.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/encryption.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/ipfs.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/zkps.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/drift.js
(base) nic@Proto-forge:~/identity-vault$ nano utils/integrations.js
(base) nic@Proto-forge:~/identity-vault$ mkdir routes
(base) nic@Proto-forge:~/identity-vault$ nano routes/profiles.js
(base) nic@Proto-forge:~/identity-vault$ nano routes/feedback.js
(base) nic@Proto-forge:~/identity-vault$ nano routes/agents.js
(base) nic@Proto-forge:~/identity-vault$ nano cli.js
(base) nic@Proto-forge:~/identity-vault$ nano .env.example
(base) nic@Proto-forge:~/identity-vault$ nano .gitignore
(base) nic@Proto-forge:~/identity-vault$ nano README.md
(base) nic@Proto-forge:~/identity-vault$ mkdir templates
(base) nic@Proto-forge:~/identity-vault$ nano templates/user-template.json
(base) nic@Proto-forge:~/identity-vault$ nano templates/agent-template.json
```

```
(base) nic@Proto-forge:~/identity-vault$ mkdir plugins
(base) nic@Proto-forge:~/identity-vault$ nano plugins/example-hook.js
(base) nic@Proto-forge:~/identity-vault$ mkdir tests
(base) nic@Proto-forge:~/identity-vault$ nano tests/server.test.js
(base) nic@Proto-forge:~/identity-vault$ npm install dotenv cors body-parser ipfs-core snarkjs
json-diff @langchain/core multiformats commander ngrok chalk axios
npm warn deprecated datastore-fs@8.0.0: This package is deprecated. Instead, use
datastore-level in Node.js, and datastore-idb in browsers
npm warn deprecated ipfs-core-utils@0.18.1: js-IPFS has been deprecated in favour of Helia -
please see https://github.com/ipfs/js-ipfs/issues/4336 for details
npm warn deprecated ipfs-core-config@0.7.1: js-IPFS has been deprecated in favour of Helia -
please see https://github.com/ipfs/js-ipfs/issues/4336 for details
npm warn deprecated ipfs-core-types@0.14.1: js-IPFS has been deprecated in favour of Helia -
please see https://github.com/ipfs/js-ipfs/issues/4336 for details
npm warn deprecated ipfs-http-client@60.0.1: js-IPFS has been deprecated in favour of Helia -
please see https://github.com/ipfs/js-ipfs/issues/4336 for details
npm warn deprecated @libp2p/mplex@7.1.7: Mplex has no flow control - please use
@chainsafe/libp2p-yamux instead
npm warn deprecated ipfs-core@0.18.1: js-IPFS has been deprecated in favour of Helia -
please see https://github.com/ipfs/js-ipfs/issues/4336 for details

added 746 packages, and audited 928 packages in 22s

80 packages are looking for funding
  run `npm fund` for details

17 vulnerabilities (2 moderate, 15 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
(base) nic@Proto-forge:~/identity-vault$ npm install --save-dev jest supertest nodemon

added 348 packages, and audited 1276 packages in 14s

124 packages are looking for funding
  run `npm fund` for details

17 vulnerabilities (2 moderate, 15 high)

To address issues that do not require attention, run:
  npm audit fix
```

```
To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
(base) nic@Proto-forge:~/identity-vault$ npm run dev
npm error Missing script: "dev"
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in:
/home/nic/.npm/_logs/2025-09-15T01_40_44_565Z-debug-0.log
(base) nic@Proto-forge:~/identity-vault$ nano package.json
(base) nic@Proto-forge:~/identity-vault$ cat package.json
{
  "name": "identity-vault",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js",
  "test": "jest",
  "cli": "node cli.js",
  "generate-templates": "node cli.js generate-templates"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "@langchain/core": "^0.3.75",
    "axios": "^1.12.2",
    "body-parser": "^2.2.0",
    "chalk": "^5.6.2",
    "commander": "^13.1.0",
    "cors": "^2.8.5",
    "dotenv": "^17.2.2",
    "express": "^5.1.0",
    "ipfs-core": "^0.18.1",
    "json-diff": "^1.0.6",
    "multiformats": "^13.4.0",
    "ngrok": "^5.0.0-beta.2",
    "snarkjs": "^0.7.5",
    "sqlite3": "^5.1.7"
  },
```

```
  "devDependencies": {
    "jest": "^30.1.3",
    "nodemon": "^3.1.10",
    "supertest": "^7.1.4"
  }
}
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: ⚙️  enable debug logging with { debug: true }
node:internal/modules/cjs/loader:1143
  throw err;
  ^

Error: Cannot find module '../utils/encryption'
Require stack:
- /home/nic/identity-vault/db.js
- /home/nic/identity-vault/server.js
    at Module._resolveFilename (node:internal/modules/cjs/loader:1140:15)
    at Module._load (node:internal/modules/cjs/loader:981:27)
    at Module.require (node:internal/modules/cjs/loader:1231:19)
    at require (node:internal/modules/helpers:177:18)
    at Object.<anonymous> (/home/nic/identity-vault/db.js:11:30)
    at Module._compile (node:internal/modules/cjs/loader:1364:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1422:10)
    at Module.load (node:internal/modules/cjs/loader:1203:32)
    at Module._load (node:internal/modules/cjs/loader:1019:12)
    at Module.require (node:internal/modules/cjs/loader:1231:19) {
  code: 'MODULE_NOT_FOUND',
  requireStack: [
    '/home/nic/identity-vault/db.js',
    '/home/nic/identity-vault/server.js'
  ]
}

Node.js v18.20.8
[nodemon] app crashed - waiting for file changes before starting...
```

```
^X^C
(base) nic@Proto-forge:~/identity-vault$ nano db.js
(base) nic@Proto-forge:~/identity-vault$ grep -n "encryption" db.js
4:// encryption passthrough (store encrypted blobs), multi-agent session state, regulatory
exports.
11:const { encrypt, decrypt } = require('./utils/encryption'); // Passthrough for at-rest encryption
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: 🔐 prevent committing .env to code:
https://dotenvx.com/precommit
/home/nic/identity-vault/utils/zkps.js:36
      vk_alphabeta_12: [['0x...', '0x...'], ...] // Full stub omitted for brevity
                                       ^

SyntaxError: Unexpected token ']'
    at internalCompileFunction (node:internal/vm:76:18)
    at wrapSafe (node:internal/modules/cjs/loader:1283:20)
    at Module._compile (node:internal/modules/cjs/loader:1328:27)
    at Module._extensions..js (node:internal/modules/cjs/loader:1422:10)
    at Module.load (node:internal/modules/cjs/loader:1203:32)
    at Module._load (node:internal/modules/cjs/loader:1019:12)
    at Module.require (node:internal/modules/cjs/loader:1231:19)
    at require (node:internal/modules/helpers:177:18)
    at Object.<anonymous> (/home/nic/identity-vault/utils/drift.js:15:23)
    at Module._compile (node:internal/modules/cjs/loader:1364:14)

Node.js v18.20.8
[nodemon] app crashed - waiting for file changes before starting...
^X^C
(base) nic@Proto-forge:~/identity-vault$ nano utils/zkps.js
(base) nic@Proto-forge:~/identity-vault$ grep -n "vk_alphabeta_12" utils/zkps.js
36:      vk_alphabeta_12: [['0x1', '0x2'], ['0x3', '0x4']] // Stub vectors for VK
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js
```

```
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: ⚙️  suppress all logs with { quiet: true }
node:internal/modules/cjs/loader:597
      throw e;
      ^

Error [ERR_PACKAGE_PATH_NOT_EXPORTED]: No "exports" main defined in
/home/nic/identity-vault/node_modules/ipfs-core/package.json
    at new NodeError (node:internal/errors:405:5)
    at exportsNotFound (node:internal/modules/esm/resolve:366:10)
    at packageExportsResolve (node:internal/modules/esm/resolve:656:13)
    at resolveExports (node:internal/modules/cjs/loader:590:36)
    at Module._findPath (node:internal/modules/cjs/loader:664:31)
    at Module._resolveFilename (node:internal/modules/cjs/loader:1126:27)
    at Module._load (node:internal/modules/cjs/loader:981:27)
    at Module.require (node:internal/modules/cjs/loader:1231:19)
    at require (node:internal/modules/helpers:177:18)
    at Object.<anonymous> (/home/nic/identity-vault/utils/ipfs.js:12:20) {
  code: 'ERR_PACKAGE_PATH_NOT_EXPORTED'
}

Node.js v18.20.8
[nodemon] app crashed - waiting for file changes before starting...
^C
(base) nic@Proto-forge:~/identity-vault$ npm uninstall ipfs-core multiformats

added 1 package, removed 611 packages, and audited 666 packages in 2s

90 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
(base) nic@Proto-forge:~/identity-vault$ nano utils/ipfs.js
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
```

```
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: 🔐 prevent building .env in docker:
https://dotenvx.com/prebuild
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^X^CShutting down Vault...
/home/nic/identity-vault/server.js:174
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:174:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ mkdir public
(base) nic@Proto-forge:~/identity-vault$ nano public/ui.html
(base) nic@Proto-forge:~/identity-vault$ nano server.js
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: 🛠️  run anywhere with `dotenvx run --
yourcommand`
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^
```

```
ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ nano public/ui.html
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: ⚙️  override existing env vars with { override:
true }
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ nano public/ui.html
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
```

```
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (0) from .env -- tip: 📡 auto-backup env with Radar:
https://dotenvx.com/radar
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ cp .env.example .env
(base) nic@Proto-forge:~/identity-vault$ nano .env
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (14) from .env -- tip: 📡 version env with Radar:
https://dotenvx.com/radar
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^X^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
```

```
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ nano .env
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (14) from .env -- tip: ⚙️  enable debug logging with { debug: true }
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ nano .env
(base) nic@Proto-forge:~/identity-vault$ npm run dev

> identity-vault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.2.2] injecting env (14) from .env -- tip: 🔐 prevent building .env in docker:
https://dotenvx.com/prebuild
```

```
Identity Vault running at http://localhost:3000
Endpoints: /health, /profiles/create/:type, /profiles/:type/:id, etc.
Advanced: ZKP for trust, IPFS for P2P, drift tracking enabled.
DB Initialized: Profiles, logs, and state tables ready.
^X^CShutting down Vault...
/home/nic/identity-vault/server.js:176
  db.close((err) => {
  ^

ReferenceError: db is not defined
    at process.<anonymous> (/home/nic/identity-vault/server.js:176:3)
    at process.emit (node:events:517:28)

Node.js v18.20.8

(base) nic@Proto-forge:~/identity-vault$ git remote add origin
https://github.com/BogaertN/identity-vault.git
git branch -M main
git push -u origin main
error: remote origin already exists.
error: src refspec main does not match any
error: failed to push some refs to
'https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git'
(base) nic@Proto-forge:~/identity-vault$ echo "# identity-vault" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/BogaertN/identity-vault.git
git push -u origin main
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:     git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:     git branch -m <name>
Initialized empty Git repository in /home/nic/identity-vault/.git/
[master (root-commit) d527265] first commit
 1 file changed, 142 insertions(+)
 create mode 100644 README.md
```

```
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 24 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 2.98 KiB | 2.98 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/BogaertN/identity-vault.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(base) nic@Proto-forge:~/identity-vault$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env.example
        .gitignore
        cli.js
        db.js
        package-lock.json
        package.json
        plugins/
        public/
        routes/
        schemas.js
        server.js
        templates/
        tests/
        utils/

nothing added to commit but untracked files present (use "git add" to track)
(base) nic@Proto-forge:~/identity-vault$ git add .
(base) nic@Proto-forge:~/identity-vault$ git commit -m """
[main 3e134cf] '
 22 files changed, 12581 insertions(+)
 create mode 100644 .env.example
 create mode 100644 .gitignore
 create mode 100644 cli.js
 create mode 100644 db.js
 create mode 100644 package-lock.json
 create mode 100644 package.json
 create mode 100644 plugins/example-hook.js
 create mode 100644 public/ui.html
 create mode 100644 routes/agents.js
```

```
 create mode 100644 routes/feedback.js
 create mode 100644 routes/profiles.js
 create mode 100644 schemas.js
 create mode 100644 server.js
 create mode 100644 templates/agent-template.json
 create mode 100644 templates/user-template.json
 create mode 100644 tests/server.test.js
 create mode 100644 utils/drift.js
 create mode 100644 utils/encryption.js
 create mode 100644 utils/integrations.js
 create mode 100644 utils/ipfs.js
 create mode 100644 utils/prompts.js
 create mode 100644 utils/zkps.js
(base) nic@Proto-forge:~/identity-vault$ git push origin main
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 24 threads
Compressing objects: 100% (27/27), done.
Writing objects: 100% (30/30), 122.11 KiB | 13.57 MiB/s, done.
Total 30 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/BogaertN/identity-vault.git
   d527265..3e134cf  main -> main
(base) nic@Proto-forge:~/identity-vault$
```

Tab 4

```
{
  "user_id": "nic_bogaert",
  "canonical_name": "Nicholas Jacob Bogaert",
  "spirit_name": "Manitou Benishi",
  "project_affiliations": [
    "AI.Web",
    "ProtoForge",
    "EchoForge",
    "Gilligan",
    "Athena",
    "Neo"
  ],
  "identity_tags": [
    "symbolic_builder",
    "recursive_founder",
    "meta-cognitive_engineer",
    "truth-seeker",
    "critical_thinker",
    "philosophical_synthesist"
  ],
  "version": "1.0.0",
  "last_updated": "2025-09-12T03:00:00Z",

  "project_context": {
    "current_project": "AI.Web",
    "phase": "Meta-Insight Synthesis",
    "current_files": [
      "AI.Web – A New Operating System for Human Thought.pdf",
      "ProtoForge Volume II – The Full System Dashboard.pdf"
    ],
    "active_collaborators": [
      "Russell Wright",
      "Andy Morley",
      "Jenna Smith"
    ],
    "subsystems": [
      "Symbolic Memory Engine",
      "Recursive Feedback Loop",
      "Drift Detection System"
    ],
    "goals": [
      "Refactor Gilligan meta-cognition stack",
      "Document phase recursion rules",
      "Push next draft to GitHub"
```

```
  ]
},

"interaction_preferences": {
  "formality": "stoic",
  "depth": "maximum",
  "step_by_step": true,
  "beginner_friendly": true,
  "plain_language": true,
  "no_boxes": true,
  "pushback": true,
  "critique_mandatory": true,
  "confirmation_required": true,
  "formatting_rules": [
    "never use tables or boxed content",
    "always provide explicit field explanations for JSON or API outputs",
    "always begin code blocks with a header comment for copy-paste"
  ]
},

"meta_rules": {
  "recursive_feedback": true,
  "drift_tracking": true,
  "contradiction_alerts": true,
  "require_honest_pushback": true,
  "preserve_session_memory": true,
  "require_context_carryover": true,
  "log_all_exchanges": true,
  "always_explain_decisions": true,
  "safe_words": [
    "pause", "stop", "reset", "meta"
  ],
  "forbidden_actions": [
    "summarize before full context is explored",
    "skip steps",
    "respond in affirmations only"
  ]
},

"session_state": {
  "phase": "longform_deep_dive",
  "waiting_for": "API formalization feedback",
  "last_feedback": "Ready to deploy new operational API. Next: document usage.",
  "last_action": "Confirmed operational identity template delivery.",
```

```
    "timestamp": "2025-09-12T03:01:00Z"
  }
}
```